

Fakultet for Ingeniørvitenskap og Teknologi

Institutt for Datateknologi og Beregningsorientert Ingeniørfag

Prototype på kryssplattform flerspiller online spill

Bacheloroppgave sluttrapport

—

Bjerkan, Tore

Eriksen, Mikal

Reiakvam, Carlos

Bacheloroppgave i Datateknologi, <2023-Juni-06>



Tittel: Prototype på kryssplattform flerspiller online spill		Dato: < 06.06.2023 >
		Gradering: Ingen
Forfattere: Carlos Reiakvam Mikal Eriksen Tore Bjerkan		Antall sider: < 48 >
		Antall vedlegg: < 8 >
Fakultet: IVT	Institutt: IDBI	
Studieretning: Datateknologi		
Veileder: Andreas D. Jansson, Werner Farstad		
Oppdragsgiver: UIT		
Oppdragsgivers kontaktperson: Andreas D Jansson		
Sammendrag: Dette dokumentet undersøker utfordringene knyttet til utvikling av et flerspiller online spill som fungerer på tvers av plattformer i 2023. Vi går dypt inn i prosessen med å bruke Unity som rammeverk, og vurderer også fordelene og ulempene sammenlignet med en tilnærming uten bruk av spillmotor. Selv om vi valgte bort enkelte planlagte elementer som lyd og poengsystem på grunn av tidsbegrensninger, konkluderer vi med at det, med et grunnleggende nivå av forståelse for programmering, er fullt mulig å skape en prototype av et fungerende flerspiller online spill for både mobil og PC på tvers av plattformer i løpet av fem måneder.		

Abstract:

This document investigates the challenges associated with developing a multiplayer online game that operates across platforms in 2023. We delve deeply into the process of using Unity as a framework, and also assess the advantages and disadvantages compared to an approach without the use of a game engine.

Although we omitted certain planned elements such as sound and scoring system due to time constraints, we conclude that with a basic level of understanding of programming, it is entirely possible to create a prototype of a functioning multiplayer online game for both mobile and PC across platforms within five months.

Revisjonshistorie

Dato	Versjon	Beskrivelse	Forfatter
17.05.23	0.1	Sammendrag og introduksjon (1.1)	Mikal
17.05.23	0.1	Formatering av dokument/tilpassing av mal, introduksjon	Tore
19.05.23	0.1	«2D ovenfra perspektiv» (3.3)	Mikal
19.05.23	0.1	<i>Gameplay</i> begrepet, Nettverk og Unity Services	Tore
20.05.23	0.1	Nettverkstopolgi (2.4) Arbeids og Rollefordeling	Tore
21.05.23	0.1	Synkronisering av gameobjekter	Tore
21.05.23	0.1	<i>NPC/Enemy</i> adferd (2.2)	Tore
21.05.23	0.1	Utfordringer ifm. navigasjon og vårt kompromiss	Tore
23.05.23	0.1	3.4 «Tema»	Mikal
23.05.23	0.1	Menysystem	Carlos
24.05.23	0.2	Rollefordeling	Tore
24.05.23	0.2	Forsvarer vårt valg av nettverkstopologi I forhold til Design	Tore
24.05.23	0.2	Lobby	Carlos
24.05.23	0.2	Valg av teknologi og utviklingsmetode	Tore
24.05.23	0.2	La til 4.5.4 «Enemy»	Mikal
25.05.23	0.2	Trello sin rolle i arbeidsfordelingen og organisering av prosjektet	Tore
25.05.23	0.2	Samhandling og introduksjon	Tore
27.05.23	0.3	Enemy/fiende	Tore
27.05.23	0.3	Rollefordeling	Tore
27.05.23	0.2	2D ovenfra perspektiv	Tore
27.05.23	0.2	<i>Gameplay</i>	Tore
26.05.23	0.2	Vedlegg: Dokumentasjon av kode	Carlos
27.05.23	0.2	Android og touch kontroller	Tore
27.05.23	0.2	Vedlegg: Dokumentasjon av kode	Carlos
28.05.23	0.2	Prosjekt fremgangs-diagram	Mikal
28.05.23	0.2	Android, touch kontroller og kryssplattform	Tore
28.05.23	0.2	Kapittel 5: Diskusjon vedr. avvik og oppnådde mål	Tore
28.05.23	0.2	<i>Gameplay</i>	Mikal
30.05.23	0.2	Konklusjon	Tore
30.05.23	0.2	Server vs. Owner autoritativ	Tore
30.05.23	0.3	Synkronisering av lys over nett	Carlos
30.05.23	0.3	Bruk av netterksvariabler i LoadingManager	Carlos
30.05.23	0.3	Fremgang av prosjekt	Mikal
31.05.23	0.3	Nettverksytelse/Forsinkelse	Tore
31.05.23	0.3	Konklusjon og videre arbeid	Mikal
31.05.23	0.3	Kontekst og gamedesign	Tore
31.05.23	0.3	<i>Gameplay</i>	Mikal
01.06.23	0.4	Lag switch	Carlos
01.06.23	0.4	Resultater, Nettverksytelse	Tore
02.06.23	0.5	Innledning, kontekst(spillmiljø)	Tore
02.06.23	0.5	Utfordringer	Tore
02.06.23	0.6	Nettverk og Player, Skiller ut Enemy om synkronisering	Tore
02.06.23	0.6	Sammendrag/Abstract	Carlos
02.06.23	0.6	Rettinger i forhold til tilbakemeldinger fra Andreas	Tore
02.06.23	0.6	Konklusjon og videre arbeid	Mikal
03.06.23	0.6	Gameplay	Mikal
03.06.23	0.6	Introduksjon, drøfting ulike teknologier og utviklingsmetoder	Tore
03.06.23	0.6	Teori om lobby og meny	Carlos

04.06.23	0.7	Introduksjon, <i>NPC</i> adferd, utviklingsmetoder, organiserer kapitler	Tore
04.06.23	0.7	Skriver og organiserer referanser	Tore
04.06.23	0.7	Fremgang gjennom prosjektet	Mikal
04.06.23	0.7	Omskriving	Carlos
05.06.23	0.8	Omskriving og flytting av elementer	Carlos
05.06.23	0.8	Liste over akronymer og forkortelser	Mikal
05.06.23	0.9	Liste over figurer, Liste over tabeller	Mikal
05.06.23	0.9	Forord	Tore
05.06.23	1.0	Forslag til videre utvikling	Tore

FORORD

Det er med stor glede og stolthet at vi presenterer sluttrapporten for vår bacheloroppgave om utviklingen av en prototype på et kryssplattform flerspiller online spill. Prosjektet har vært en spennende reise hvor vi har utforsket og lært om spillutvikling og nettverksteknologi.

Vi valgte oppgaven fordi vi ville utfordre oss selv å se hvor mye vi kunne få til på bakgrunn av minimal erfaring fra feltet. Spillutvikling er regnet for å være vanskelig, men hvor vanskelig kan det egentlig være? Vi har alle et brennende engasjement for spill, og det eneste vi kunne stole på var innsatsvilje, nysgjerrighet og en porsjon kreativitet til å drive prosjektet fremover.

I løpet av prosjektet har vi gjennomført litteraturundersøkelser innen spilldesign-teori, klientrammeverk og spillmotorer, serverteknologi og Unity-tjenester. Dette har gitt oss et teoretisk fundament som vi brukte til å utvikle vår prototype.

Vi sto også fritt til å utvikle spillet utover betingelsene i oppgaven. Dette inkluderte spillverden, historie eller oppdrag, valg av avatar og tilpasninger, tillegg av forskjellige gjenstander og ikke-spillbare karakterer, samt flere spillområder og en "lobby" der spillerne kobler seg opp mot server.

Vi ønsker å takke våre veiledere og faglærere for deres støtte og veiledning gjennom prosjektet. Vi vil også takke våre medstudenter og venner for deres interesse og oppmuntring underveis.

Teams, 06.06.2023

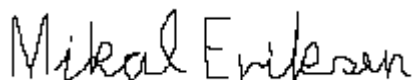
Carlos Reiakvam:



Tore Bjerkan:



Mikal Eriksen:



INNHOOLD

Forord	vi
1 Introduksjon og relevans	10
1.1 Introduksjon	10
1.2 Om Gruppen	10
1.3 Problemstilling og behov	10
1.4 Avgrensninger	10
1.4.1 Fokuspunkt	10
1.4.2 Sjanger	10
1.4.3 Begrepsavgrensninger	11
1.4.4 Plattform	11
1.5 Om rapporten	11
2 Teori: drøfting av mulige teknologier og utviklingsmetoder	11
2.1 <i>Gameplay</i> / spillopplevelse	12
2.2 NPC adferd	13
2.3 Nettverkstopologi	13
2.4 Sikkerhet og juks i online spill	16
2.4.1 Server-autoritativ og klientautoritativ	16
2.4.2 Lag-switch	17
2.5 Grafisk briukergrensesnitt spill	17
2.6 Lobby	18
2.6.1 Heartbeat	18
2.7 Systemutvikling for spillutvikling	18
2.7.1 Scrum	18
2.7.2 Kanban	18
2.7.3 Lean	18
3 Valg av teknologi og utviklingsmetode	19
3.1 Valg av spillmotor	19
3.1.1 Unity	19
3.1.2 Unreal Engine	19
3.1.3 Godot	19
3.1.4 Avgjørelse	19
3.2 Nettverk	19
3.3 Relay	20
3.4 Forbindelsen spill-design og nettverksarkitektur	21
3.5 Netcode og logikk	22

3.6	Menysystem	22
3.6.1	Reaktiv design	24
3.6.2	Tilgjengelighet.....	25
3.7	Lobbysystem.....	25
3.7.1	Lobbysystem.....	25
3.7.2	Oppkobling mot LobbyService	25
3.7.3	Lokal lobby	26
3.8	Synkronisering.....	26
3.8.1	Player	27
3.8.2	Helse til avatar.....	27
3.8.3	ChatManager	27
3.8.4	NPCs	28
3.8.5	Lys	28
3.8.6	Speed powerup	28
3.9	Mer om Enemy	28
3.10	Bytte av scener og scriptable objects	30
3.10.1	Scriptable objects	31
3.10.2	LoadingManager.....	31
3.11	Kartsystem.....	31
3.12	System for spawning av objekter over nettverk.....	32
3.13	Android og touch-kontroll	32
3.14	Arbeids og rollefordeling.....	33
3.15	grafikk, animasjon og 2D ovenfra perspektiv	34
3.16	Spillmiljø og Game Design.....	34
4	Resultater.....	36
4.1	Måloppnåelse	36
4.1.1	Nettverk og kommunikasjon.....	37
4.2	Fremgang gjennom utviklingen.....	37
4.3	Nettverksytelse.....	38
5	Diskusjon.....	39
5.1	Prosjektets avgrensning	39
5.2	definisjon av spill	40
5.3	Planlegging	40
5.4	Avvik fra opprinnelig plan og oppsatte mål	40
5.4.1	Åpnings-scene	40
5.4.2	Lobby.....	40

5.4.3	Poengsystem	41
5.4.4	Lyd	41
5.4.5	Koordinater i chat	41
5.4.6	Kompass	41
5.4.7	Stålsverd	41
5.4.8	Usynlighet på avatarer	41
5.4.9	Pause spill ved møte med objekter	42
5.4.10	Kunstig intelligens	42
5.5	Kjente bugs og problemer	42
5.5.1	Lobby	42
5.5.2	Lys	42
5.5.3	Animasjon	42
5.5.4	Fiende	42
5.5.5	Chat-input for Android	43
5.5.6	Avslutning av spill	43
5.6	Grafikk	43
5.7	nettverkskommunikasjon	43
6	Konklusjon og videre arbeid	44
7	Referanser	44
8	Liste over figurer	47
9	Liste over tabeller	47
10	Liste over akronymer og forkortelser	47
11	Vedlegg	48

1 INTRODUKSJON OG RELEVANS

1.1 INTRODUKSJON

Bacheloroppgaven omhandler utviklingen av en prototype for et flerspiller online dataspill, der spillere kan koble seg til en felles server og samhandle i en dynamisk spillverden. Vi har spesifikt rettet oppmerksomheten mot å implementere nøkkelkomponenter og funksjoner som vil styrke forståelsen av nettverks-kommunikasjon.

Oppgaven kan ha god nytte for fremtidige studenter og utviklere som vil vurdere mulig tidsbruk og realistisk oppnåelse av utvikling av et lignende spillkonsept. Oppgaven kan også gi en forståelse for hvordan nyutdannede utviklere møter nye rammeverk, og prosessen som følger av dette. Vi mener derfor at dette er en verdifull og relevant emne for en bacheloroppgave.

1.2 OM GRUPPEN

Gruppen består av tre datateknikk-studenter som benyttet våren 2023 til å dykke ned i spillutvikling. Vi er alle nettstudenter og har brukt Teams for møter, Slack for kommunikasjon og GIT for versjonskontroll. Selv om ingen av oss har en spesialisert bakgrunn innen spillutvikling, har vi tilegnet oss ferdigheter gjennom utviklingsprosessen. Vår erfaring varierer fra enkel spillutvikling med Direct-X, kjennskap til Godot, til å ta kurs i datagrafikk der spillutvikling var en del av læreplanen.

1.3 PROBLEMSTILLING OG BEHOV

Denne oppgaven undersøker hvilken effekt bruken av ulike spillverktøy har på utviklingsprosessen av en prototype på et kryssplattform, flerspiller online spill. Vi vurderer dette gjennom vår egen prosess som tre dataingeniørstudenter med minimal kjennskap til spillutvikling. Er vi i stand til å utvikle en fungerende prototype på et flerspiller, kryssplattform nettspill i løpet av fem måneder?

1.4 AVGRENSNINGER

1.4.1 Fokuspunkt

Den viktigste forutsetningen for samhandling over nett er formodentlig lav forsinkelse hos klientene. Claypool, M., and Claypool, K hevder at forsinkelse er den største barrieren i online spill

“Since most online games have a relatively low bitrate and are somewhat resistant to network loss, network latency is often the primary bottleneck for acceptable online game performance[1, s. 222].”

Vi har derfor valgt å fokusere på spillerinteraksjon og synkronisering av samhandling over nett. Samhandling er dekkende for flere av kriteriene i oppgaven.

Vi tolker interaksjon i en online spillkontekst som muligheten til å chatte, se hverandres bevegelser og status, samhandle i kamp, og overvinne utfordringer sammen. Mens spilldesign-teori, rammeverk/spillmotor og nettverkskommunikasjon er forutsetninger for samhandling i et online miljø.

1.4.2 Sjanger

Vi ønsket at spillet skulle foregå i et miljø inspirert av klassiske digitale rollespill, hvorav «Zelda» er det mest kjente [2]. Vi kan også nevne «Chrono Trigger» som i likhet med Zelda var en av de få tidlige spillene der kamp foregikk i samme vindu som navigasjonen[3].

Da vi skulle velge sjanger ble det diskutert hvilke situasjoner og utfordringer som passet best i forhold til prototypen og hvilke kompromiss vi måtte inngå for å tilpasse oppgaven til våre evner.

Etter brainstorming og forslag fra en bok i spilldesign kom vi fram til at den beste sjangeren for prototypen ville være action-eventyr. Det innebærer at spillerne kan utforske spillverden, og overkomme utfordringer i form av å navigere områder der fiender patruljerer [4, s. 71]. Fordi fokuset er på kjernen i spillopplevelsen (input og output) valgte vi 2D grafikk med lav oppløsning.

1.4.3 Begrepsavgrensninger

Vi har valgt å skille mellom begrepene 'spiller' og 'avatar' i dokumentet. I oppgaveteksten fra vår oppdragsgiver ble 'avatar' omtalt som figuren i spillet, kontrollert av spilleren. Når vi skriver spiller mener vi mennesket foran skjermen. Med avatar mener vi figuren styrt av spilleren. Oppgaven opplyser at «Spillere skal kunne se hverandres avatarer på serveren, og bevegelse skal oppdateres i «sanntid»» [Se vedlegg A for mer informasjon].

1.4.4 Plattform

Vi har valgt å utvikle for Windows og Android. Windows er et nærliggende valg fordi vi som utviklere bruker dette til daglig på våre maskiner. Android er interessant fordi det er en plattform hvor internetttilgangen ofte har lavere hastigheter gjennom for eksempel 4G. Et annet aspekt er at Android enheter ofte er mindre kraftige i forhold til maskiner som kjører Windows.

Dette gjør at vi må sikre at spillet fungerer optimalt uavhengig av plattform, og gjøre tilpasninger avhengig av internetthastighet og maskinkraft.

1.5 OM RAPPORTEN

Vi har i dette kapittelet, kapittel 1, gått gjennom problemstillingen og hvilke avgrensninger vi har satt i forhold til den.

I kapittel 2 beskriver vi ulike teorier og alternativer som er relevant for spillutvikling. I den forbindelse har vi foretatt litteratursøk, som er et nøkkelpunkt i oppgaven fra vår oppdragsgiver.

Kapittel 3 relatere vi teorien til praksis. Vi har her en gjennomgang av innholdet i prosjektet i forhold til valgene presentert i forrige kapittel. Vi går nærmere inn på Unity som spillmotor og hvordan vi løste utfordringene med å utvikle en prototype.

Kapittel 4 viser vi resultater og forteller om hvilke mål vi oppnådde. Vi går blant annet gjennom nettverksytelse for å vurdere om spillet er spillbart i forhold til våre mål.

Kapittel 5 inneholder diskusjon om problemer som oppstod underveis og hvilke funksjoner vi har valgt bort underveis.

Til slutt, i kapittel 6 vurderer vi gjennomføringsevne av planen. Vi svarer her på om det er hensiktsmessig å videreutvikle prototypen og veier svakheter opp mot det vi synes fungerer.

2 TEORI: DRØFTING AV MULIGE TEKNOLOGIER OG UTVIKLINGSMETODER

Terskelen for å produsere spill har over tid blitt stadig lavere og kravet til spesialkunnskap varierer sterkt. En bakgrunn fra programmering er i mange tilfeller ikke nødvendig likevel om det begrenser handlingsrommet.

Spiller automatikken oss et puss eller oppnår vi bedre kvalitet ved å bruke et spill-verktøy der mye foregår i en editor? Vi må erkjenne at vi opererer på et relativt høyt abstraksjonsnivå og at det ofte er ukjent hva som foregår bak fasaden.

Det er kanskje enklere å *debugge* hvis man kjenner hvordan spillet fungerer fra et fundamentalt ståsted. Ingenting er abstrahert bort og utvikleren har kontroll over alle aspekter. Ikke minst er veien til optimalisering sannsynligvis kortere når utvikleren har full oversikt.

Men i de fleste tilfeller er det opp til utviklerne selv å vurdere hvor mye automatikk de trenger. Mange spillmotorer er open source og det er mulig å tilpasse/modifisere de etter eget ønske. Det er dermed mulig å ta kontroll over alle aspektene ved utviklingen, også ved bruk av spillmotor.

2.1 GAMEPLAY / SPILLOPPLEVELSE

I våre litteraturundersøkelser i grunnleggende spilldesign har vi benyttet oss av både digitale og fysiske kilder. Antagelsen er at når forskjellige kilder vektlegger de samme temaene, indikerer dette at vi har identifisert forutsetninger for et vellykket spill. Derfor har vi forsøkt å finne fellestrekk i kildene. Flere peker på at det meste er underordnet *gameplay* eller spillopplevelse i en tidlig fase. Vi bruker det fornorskede ordet spillopplevelse om *gameplay* videre i rapporten.

Det er vanskelig å finne en universelt akseptert definisjon på spillopplevelse, men fra et brukerperspektiv er det ifølge Fabricatore tydelig hva spillere ønsker. «When playing a game, players want to be challenged, control what surround them, develop a sense of mastership and achievement, and be rewarded consequently»[5, s.5]

Da vi besluttet hvilke deler av spillet vi måtte utelate på grunn av vi ikke hadde nok tid, kuttet vi først elementer ikke relatert til kjernemekanikker i spillopplevelse. Vi fant konsensus i vårt litteratursøk for at dette var en fornuftig avgjørelse.

M. Killick poengterer at spillopplevelse er overordnet alt annet i en prototype. Bakgrunnshistorie, karakterer, våpen, menyer, lyd og perifere mekanikker har sin plass i utviklingen, men i en prototype er det viktigst for spilleren å interagere med spillverden [6, s.208]. Kjernemekanikken. Spilleren gjør noe (input) og spillverden reagerer(output).

Vi minner om at vårt mål var å lage en prototype. Prototypen skal først og fremst formidle hvilke handlinger spilleren kan foreta seg og hvordan omgivelsene svarer. Underholdningen må ikke bli overskygget av andre aspekter, uavhengig av hvor mye arbeid det er lagt i dem.

E. Adams skriver i likhet med de andre at spillopplevelse er essensielt.

Gameplay comes first. Before all other considerations, create your game to give people fun things to do. A good many games aren't fun because the designers spent more time thinking about their graphics or their story than they did thinking about creating gameplay. [4, s. 252]

Det er også viktig at en spilldesigner setter seg inn i målgruppens tankesett og perspektiv, for å forstå hvilke kjernemekanikker som passer best.

In the player-centric approach, this means learning to think like your intended players, whoever they may be: little girls, old men, busy mothers, and so on. You cannot assume that players like what you like. Rather, you must learn to design for what they like. [4, s. 31]

Å gjøre det vanskelig å gjøre fremgang i spillet er ikke nødvendigvis den beste måten å gjøre spillet underholdende på. Det er en balansegang og det er fort gjort at spillet blir frustrerende. Man må være bevisst på at det ikke lages utfordringer bare for å gjøre spillet vanskeligere, men at disse samtidig må bidra til å forbedre spillopplevelsen. Det er viktig at spilldesigneren ikke bare ser på spillere som motstandere som må forhindres fra å nå et mål. [4, s. 33]

Et annet aspekt ved spillopplevelse er kvaliteten på kontrollere. Er de naturlig plassert? Hvor responsive er de i forhold til forsinkelse på nett. Er kontrollene logisk for spilleren? Innretningen på

kontrollene bør ikke påvirke vanskelighetsgraden i spillet. Det kan skape frustrasjon uansett hvor bra det øvrige spillet er.

I avgrensningen i første kapittel fortalte vi at vi skulle utvikle for flere plattformer, mobil og Windows. Windows-spill har over tid opparbeidet seg en standard, og dermed mindre rom for hva spillerne aksepterer. WASD forventes i spill der spilleren skal styre en avatar. Regimet har oppstått som et resultat av ergonomi og nærhet til andre taster. I motsetning til piltastene som er for seg selv. Mange kjente utgivelser bruker dette oppsettet [7].

Mobil har ikke like lang fartstid. Men i sjangeren action-eventyr begynner det å bli vanlig med digital joystick og en digital knapp nederst i hvert sitt hjørne av skjermen [8]. Da det kjente spillet «Diablo immortal» kom til mobil ble kontrollene plassert nederst på hver side av skjermen [9].

2.2 NPC ADFERD

Et sentralt aspekt er adferd eller kunstig intelligens hos ikke-spillende karakterer (heretter kalt *NPCs*). Det er vanlig at *NPCs* har rutiner eller mønstre som gjør de troverdige i forhold til sin rolle. Å implementere kunstig intelligens for navigasjon i spillverden kan være en kompleks oppgave, og det finnes flere populære algoritmer som brukes til å løse dette problemet

A^* er en kjent algoritme som finner korteste vei til målet basert på gitte betingelser, der figuren navigerer rundt hindringer. Underveis gjør figuren flere beregninger i forhold til hvor den er, hva som er i nærheten og hvor den skal. Den garanterer å finne den korteste veien mellom to punkter, gitt at en vei eksisterer. En ulempe er at den kan være ressurskrevende, spesielt på store kart med mange hindringer.

Dijkstra's Algoritme er en enklere form for navigasjon, som fungerer ved å systematisk utforske alle mulige veier fra et startpunkt til et mål er nådd, og er mindre komplisert enn A^* .

Det er en god del raskere med forhåndsgenererte grafer, som beskriver alle gyldige ruter som AI-figuren kan velge blant. Det kan imidlertid virke mindre troverdig. Det kommer an på hvilken rolle *NPCs* har i forhold til rutiner. Forhåndsgenererte ruter tar hensyn til størrelsen på et område, men algoritmene trenger ikke å vite størrelsen på kartet på forhånd. Denne løsningen krever mer arbeid ved opprettelse av kart, da hvert område må defineres manuelt. Det er også mulig å bruke grafer i sammenheng med nevnte algoritmer.

En alternativ tilnærming er å lage en *statemachine* eller tilstandsmaskin, der *NPCs* bytter tilstand etter hvert som betingelsene skifter. Det er opp til utvikleren å avgjøre hvilke betingelser som utløser forskjellige tilstander. Eksempelvis kan en *NPC* kokk bytte mellom å lese oppskrifter, lage mat, gå for å servere maten og returnere. Dette er en svært deterministisk tilnærming, men gir også en illusjon av intelligent oppførsel. Ingen av de ovenfornevnte løsningene utelukker de andre og kan brukes i kombinasjon.

2.3 Nettverkstopologi

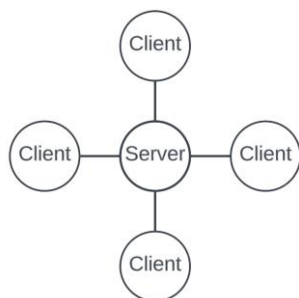
Valg av nettverkstopologi/arkitektur må avgjøres tidlig. Valget er basert på en rekke faktorer. Det gjelder spill-kategori, antall spillere, ytelses-behov, geografisk spredning av spillere og tekniske ferdigheter til utviklerne.

K. Raaen vektlegger at spill-kategori er avgjørende for hvor mye forsinkelse spillere aksepterer. Vi antar at dette også gjelder vårt spill til tross for at spillet ikke er direkte sammenlignbart med noen av de spillene Raaen nevner [10].

Eksempler på spill-kategorier som stiller ulike krav til nettverksarkitektur:

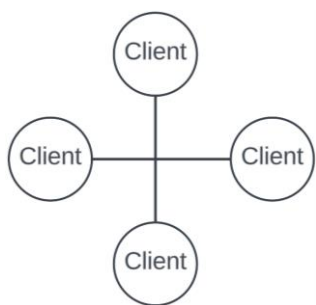
- Simulering av store slag i et intensivt og massivt krigsspill. Mange hundre spillere deltar på samme kart og er delt inn i lag som kjemper mot hverandre. Spillet foregår i sanntid og spillere som reagerer raskt blir belønnet. I et slikt scenario er det hensiktsmessig å investere i ytelse.
- Et rollespill med begrenset antall spillere i samme spillverden. Fokus er på samarbeid og løsning av gåter. Hvis man er nødt til å kjempe mot *NPCs* går spillet over i taktisk modus som ikke foregår i sanntid. Men går på runder der hver spiller venter på sin tur til å handle. *NPCs* inngår også i denne mekanikken og må vente på sin tur til å handle, på lik linje med spillerne. I dette tilfellet er ikke behovet for ytelse like opplagt som i forrige eksempel, og man trenger ikke å vektlegge investeringer i ytelse i like stor grad.

De vanlige nettverkstopologiene er



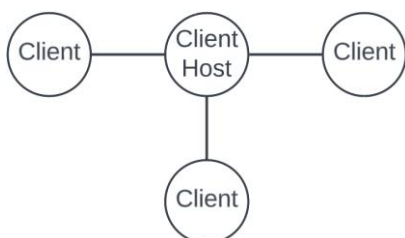
Figur 1 klient-server

Klient-server: I dette oppsettet fungerer en sentral server som den autoritative kilden til spillverdenen. Klientene kobler seg til serveren for å sende og motta spill-data. Serveren håndterer spill-logikk, synkronisering og behandler spillernes handlinger. Denne topologien gir en sentralisert kontrollmekanisme, men kan føre til forsinkelser hvis serveren er langt unna spilleren. Det er ofte sånn at man deler inn i geografiske regioner for å unngå problemet med at noen spillere befinner seg langt unna serveren. Sikkerheten er ofte godt ivaretatt av eierne av serveren.



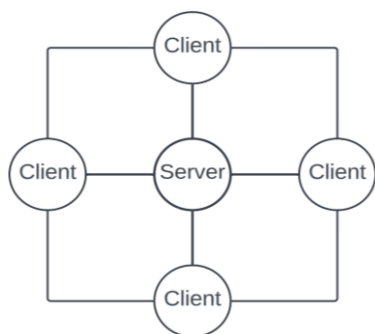
Figur 2 P2P

Peer-to-peer (P2P): I et P2P-nettverk kobler spillere seg direkte til hverandre uten en sentral server. Hver spiller fungerer samtidig som klient og server, slik at de kan utveksle spill-data direkte. P2P kan være nyttig for spill med et mindre antall spillere eller distribuerte spillverdener. Det kan imidlertid være mer utfordrende å opprettholde synkronisering og hindre juks i P2P-oppsett, siden hver maskin har total kontroll på hva den deler med andre maskiner.



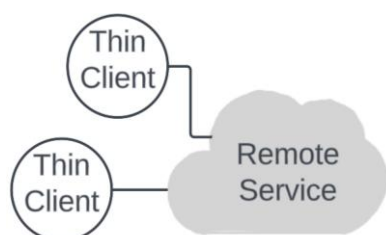
Figur 3 Hybrid

Hybrid: Noen online spill bruker en kombinasjon av klient-server og P2P-arkitekturer. Den sentrale serveren håndterer viktige spill-aspekter som spillerautentisering, matchmaking og spill-progresjon, mens individuelle spill-elementer bruker P2P-tilkoblinger. Denne hybridtilnærmingen har som mål å balansere fordelene med begge topologiene, med hensyn til skaler-barhet, kontroll og redusert serverbelastning.



Figur 4 Mesh

Mesh: I visse tilfeller, spesielt for massivt flerspiller online-spill (MMO-spill), kan en mesh-nettverkstopologi brukes. I denne topologien kobler hver spillers enhet seg til flere andre enheter, og skaper et desentralisert nettverk der spill-data kan overføres gjennom flere linjer. Dette kan forbedre feiltoleranse, redusere avhengigheten av en enkelt server og øke skalerbarheten.



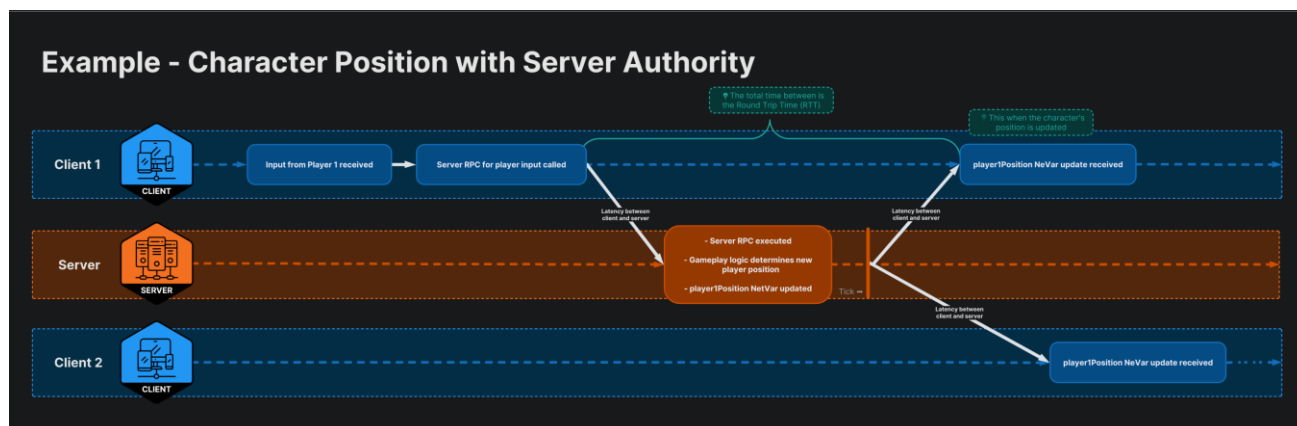
Figur 5 Sky-basert

Sky-basert: Med økningen av skybaserte spilltjenester bruker noen online spill en sky-basert nettverkstopologi. I denne oppsettet kjøres instanser av spill-servere i skyen, slik at spillere kan koble seg til og samhandle med spillet eksternt. Skybaserte spill kan gi skaler-barhet, global tilgjengelighet og reduserte maskinvarekrav for spillerne. Fordelen er at det ikke stiller høye maskinkrav til klienten. Ulempen er at klienten er avhengig av høyhastighet bredbånd og må betale for tjenesten.

2.4 SIKKERHET OG JUKS I ONLINE SPILL

2.4.1 Server-autoritativ og klientautoritativ

Server-autoritativ betyr at enhver forandring hos en klient blir sendt til server før server distribuerer meldingen til alle klienter. Alle avgjørelser blir tatt sentralt hos server. Det er to store fordeler med server-autoritativ tilnærming. En fordel er konsistens slik at hendelser inntreffer samtidig hos klienter. Den andre er sikkerhet. Server sjekker alle verdiene sendt av klienter slik at de stemmer overens med spillets regler.



Bilde 1 Hentet fra dokumentasjonen av Unity NetCode men prinsippet gjelder generelt [11]

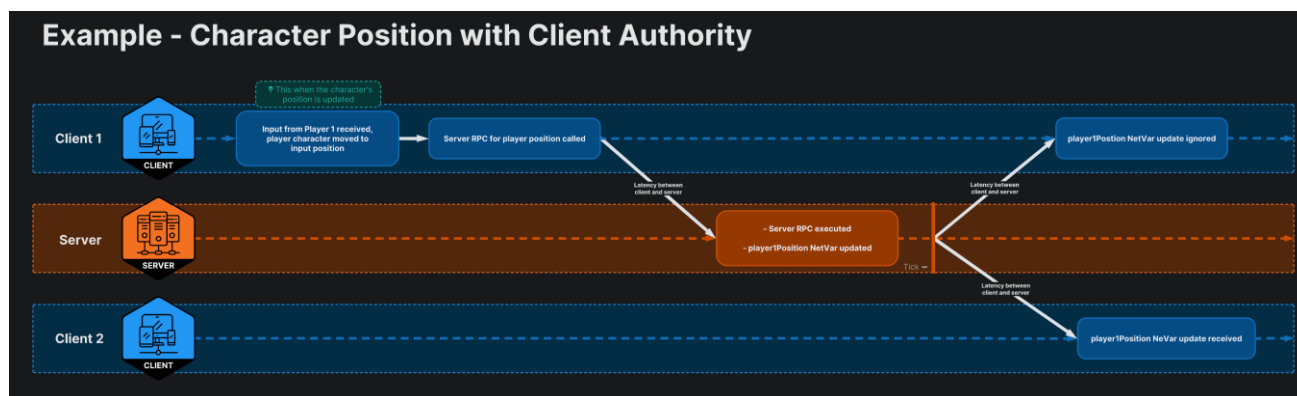
En nedside med server-autoritativ løsning er reaktivitet. Klientene må vente til serveren gir beskjed om å oppdatere spillstatus. Når klienten sender input til serveren for å oppdatere posisjon på spillfiguren, må klienten vente tiden det tar å få sendt forespørselen til server, mottar og behandler den og sender den i retur (RTT). Først når klienten mottar svar fra server kan den oppdatere posisjonen. Dette tar lang tid og spillerne vil sannsynligvis gi opp spillet når det oppstår mye forsinkelse.

Veien rundt problemet er vanligvis en form for prediksjon slik at transformasjonene oppfattes som momentane. Det innebærer at klienten flytter objektet til den mest sannsynlige posisjonen mens den venter på at server skal bekrefte posisjonen. En illusjon, men dersom spilldesignet er deterministisk er dette oppnåelig. Deterministisk vil si at om man har en game state og et sett med input vil resultatet være forutsigbart.

I de mest tids-sensitive PVP-spillene synes opplevelsen å degraderes når forsinkelsen mellom input og output overstiger 50 ms [10, s. 1].

I motsatt fall i et klientautoritativt design er det fortsatt server som distribuerer informasjon om hendelser i spillverden, men klientene vil eie og påvirke sin egen virkelighet lokalt. Dette betyr mye for reaktivitet. En klient får øyeblikkelig respons på sine egne handlinger siden den eier sin egen virkelighet. Serveren må fortsatt formidle hendelsen til øvrige klienter, men klienten slipper å vente på

å se oppdateringer av egne handlinger i sin egen verden. Dette vil også belaste serveren mindre siden klientene behandler input lokalt.



Bilde 2 er hentet fra dokumentasjon av Unity NetCode men prinsippet gjelder generelt [12]

Den store ulempen med klientautoritativt design er at den åpner for juks. Når en klient eier sin egen virkelighet kan den sende input uten at det blir foretatt kontroll. Serveren bare videreformidler. Den regner de oppdateringene klienten gjør i sin verden for å være sannferdige.

2.4.2 Lag-switch

Lag-switch er en teknikk som spillere bruker til å utnytte deres egen internettforbindelse for å skape fordeler for seg selv. Dette skjer ved at de forstyrrer datastrømmen mellom deres maskin og spillserveren, noe som kan resultere i at deres handlinger oppfattes forsinket av de andre spillerne. Denne urettferdige fordelingen kan ødelegge spillernes opplevelse og skape ubalanse i spillet.

Unity har noen verktøy som kan hjelpe spillutviklere med å motvirke denne problemstillingen. En av løsningene er nettverksinterpolering. Denne teknikken gjør det mulig å glatte ut plutselige endringer i spillernes posisjoner ved å forutsi bevegelsene mellom mottatte oppdateringer. På denne måten kan effekten av lag-switching reduseres.

En annen nyttig tilnærming er å bruke server-autorisasjon, hvor serveren har full kontroll over sentrale spillegenskaper. Når viktige deler av spilllogikken er i hendene på serveren, blir det mye vanskeligere for spillere å manipulere spillet ved å manipulere sin egen internettforbindelse. For eksempel, i et skytespill, kan serveren avgjøre om et skudd har truffet en annen spiller, uansett hva lag-switcherens spillklient sier.

Selv om disse teknikkene er tilgjengelige, er det ingen perfekte løsninger på lag-switching. Det er alltid en balansegang mellom å oppnå effektivitet, rettferdighet og robusthet mot slike misbruk. Men, ved å utnytte de forskjellige nettverksfunksjonene som Unity tilbyr på en god måte, kan spillutviklere jobbe mot å minimere den negative påvirkningen fra lag-switching.

2.5 GRAFISK BRUKERGRENSESNITT SPILL

Grafisk brukergrensesnitt (GUI) er en sentral del av alle spill. Det inkluderer alt som spilleren ser og interagerer med på skjermen, fra menyer til helsebarer og minikart. Menysystemet er et kritisk element av GUI som tjener som et viktig kontaktpunkt mellom spilleren og spillet.

I spill fungerer menyer ofte som portalen til ulike aspekter av spillet, som å starte eller lagre et spill, justere innstillinger, eller bli med i flerspillerspill. Et velutformet menysystem bør være intuitivt og lett å navigere, slik at spillere kan få tilgang til funksjonene de trenger uten forvirring eller frustrasjon.

Generelt sett, skal et godt GUI i et spill være tydelig, responsivt og informativt, og samtidig minimalt forstyrrende for selve spillopplevelsen. Det bør gi nødvendig informasjon og kontroller på en tydelig og

konsistent måte, slik at spillerne kan fokusere på selve spillet, i stedet for å prøve å forstå hvordan man interagerer med det.

2.6 LOBBY

Et lobbysystem, ofte referert til bare som en "lobby", er et kritisk element i mange nettspill, spesielt de som involverer flerspiller- og gruppespillmekanikker. Lobbyen tjener flere formål som hjelper til med spillerinteraksjoner og sørge for at alle er klare til å begynne å spille når spillet starter.

I sin enkleste form kan man tenke på en lobby som et slags virtuelt venterom. Dette er stedet hvor spillere samles før de går inn i selve spillet. Spillere kan komme inn i lobbyen individuelt eller som en del av en forhåndsdefinert gruppe. Når de er i lobbyen, kan de se hvem de vil spille med, velge spesifikke innstillinger for spillet, som kart eller spillmodus, og gjøre eventuelle nødvendige forberedelser.

For mange spill er lobbyen stedet hvor spillerne velger eller bekrefter deres karakterer, utstyr eller andre tilpasningsalternativer. Dette kan være en viktig del av strategien for spillet, og lobbyen gir spillere muligheten til å gjøre disse valgene i sitt eget tempo, uten presset fra å være midt i selve spillet.

Lobbyer kan også inkludere sosiale funksjoner, som chat-systemer, vennelister eller muligheten til å danne permanente lag eller "klaner". Disse funksjonene hjelper med å bygge samfunnet rundt spillet, og gir spillerne en sjanse til å diskutere strategier, dele tips, eller bare sosialisere mellom spillene.

Lobbyen spiller også en nøkkelrolle i å matche spillere for flerspillerpartier. I mange spill er det en form for matchmaking-system i lobbyen, som automatisk sammenkobler spillere basert på forskjellige kriterier, som ferdighetsnivå, geografisk plassering, eller til og med spillestil.

2.6.1 Heartbeat

"Heartbeat" er en kritisk funksjon i mange lobbysystemer. Den bekrefter at en bruker er aktiv og at lobbyen fortsatt er nødvendig. Etter at en lobby er opprettet, må brukeren sende et "heartbeat"-signal regelmessig, ofte hvert 30. sekund. Hvis ikke, antas det at lobbyen ikke lenger er i bruk og den lukkes automatisk. Dette forhindrer akkumulering av inaktive lobbyer, noe som kunne belastet systemet unødvendig og gjøre brukeropplevelsen dårligere.

2.7 SYSTEMUTVIKLING FOR SPILLUTVIKLING

Systemutviklingsmetoder blir ofte brukt innenfor mange ulike teknologiske felt, inkludert spillutvikling. Her følger en kort beskrivelse av noen av metodene vi har vurdert som egnet for oss.

2.7.1 Scrum

I spillutvikling er det vanlig med endringer i design og funksjoner underveis. Scrum er lagbasert og iterativ, delt inn i "sprinter" av fast varighet. Etter hver sprint er det en gjennomgang hvor laget vurderer hva som ble oppnådd og hva som kan forbedres. Dette tillater kontinuerlig forbedring og tilpasning, noe som er ideelt for en dynamisk og kreativ prosess som spillutvikling.

2.7.2 Kanban

Dette er en metode som fokuserer på kontinuerlig levering, og visualiserer arbeidsflyt med en Kanban-tavle. Tavlen er delt inn i kolonner som viser forskjellige stadier av prosessen, som "To Do", "In Progress", og "Done". For et spillutviklingsteam kan Kanban gi en klar oversikt over prosessen, hjelpe med å identifisere flaskehalser, og fordele arbeidsbelastningen mer effektivt.

2.7.3 Lean

Lean-utvikling handler om å levere mest mulig verdi med minst mulig sløsing av tid og ressurser. Det kan innebære å fokusere på de viktigste funksjonene i spillet først, og deretter gradvis legge til nye

funksjoner basert på faktisk spillerfeedback og etterspørsel. På denne måten kan Lean sikre at ressursene brukes mest effektivt og at utviklingsteamet holder fokus på det som gir mest verdi for spillerne.

3 VALG AV TEKNOLOGI OG UTVIKLINGSMETODE

I lys av den angitte avgrensningen, der prototypen ble bestemt å være et action-eventyrspill med fokus på spillerutforskning og navigering i områder der fiender patruljerer, samt bruk av 2D-grafikk med lav oppløsning, går vi videre med å diskutere ulike utviklingsverktøy og deres fordeler og ulemper.

3.1 VALG AV SPILLMOTOR

3.1.1 Unity

Unity bruker C# til kodespråk, som er verdifullt å lære seg, fordi det også er brukt innenfor .net universet. Nybegynnervennlig læringskurve. God støtte for online spill med eget bibliotek for nettverkslogikk, som abstraherer bort kompleksiteter med tanke på lav nivå protokoller og nettverksinfrastruktur. Godt egnet til prototyping og har et intuitivt grensesnitt. En av ulempene kan være at Unitys 2D-verktøy ikke er like robuste som deres 3D-verktøy, og noen ganger kan det være nødvendig å implementere tilpasninger eller finne alternative løsninger for å oppnå ønsket resultat.

3.1.2 Unreal Engine

Unreal Engine er kjent for sine imponerende grafiske funksjoner og kraftige verktøy, men har også noen ulemper. En av ulempene kan være at Unreal Engine er mer ressurskrevende enn andre verktøy, spesielt når det gjelder systemkrav og ytelsesoptimalisering. Dette kan være en utfordring når man jobber med 2D-grafikk og lavoppløsningsmiljøer. I tillegg kan Unreal Engine ha en brattere læringskurve sammenlignet med andre verktøy.

3.1.3 Godot

Mens Godot er et stadig mer populært og brukervennlig spillutviklingsverktøy, har det også noen ulemper å ta i betraktning. En av ulempene kan være at til tross for sin voksende popularitet, har Godot mindre support sammenlignet med Unity og Unreal Engine. Dette kan påvirke tilgjengeligheten av dokumentasjon, opplæringsressurser og ekstra funksjonalitet. I tillegg har Godot tidligere hatt begrensede muligheter for nettverksfunksjonalitet.

3.1.4 Avgjørelse

Etter en grundig vurdering av utviklingsverktøyene og deres fordeler og ulemper sett i lys av prosjektets avgrensninger, har vi bestemt oss for å bruke Unity som det foretrukne verktøyet for å utvikle prototypen. Til tross for noen potensielle ulemper, vurderer vi Unity som det beste alternativet for å oppfylle prosjektets mål og nå ut til et bredt publikum, takket være dets allsidighet, støtte for 2D-grafikk, omfattende dokumentasjon og et stort utviklersamfunn.

3.2 Nettverk

Unity tilbyr et komplett økosystem innenfor spillutvikling, som betyr at nettverkstjenestene fra Unity er godt tilpasset spillmotoren. Tanken var å unngå å bruke unødvendig tid på å integrere eksterne tjenester fra andre aktører i et univers som allerede hadde de verktøyene vi trengte. Det bidro til et ryddig utviklingsmiljø der vi unngikk å måtte forholde oss til mer enn en tilbyder av tjenester. Også med tanke på support.

En annen sentral avgjørelse var valg av nettverkstopologi. Vi presenterte noen av mulighetene i forrige kapittel. Vi landet på en hybrid løsning, der en av maskinene til spillerne fungerer som host. Host betyr at maskinen fungerer både som server og klient.

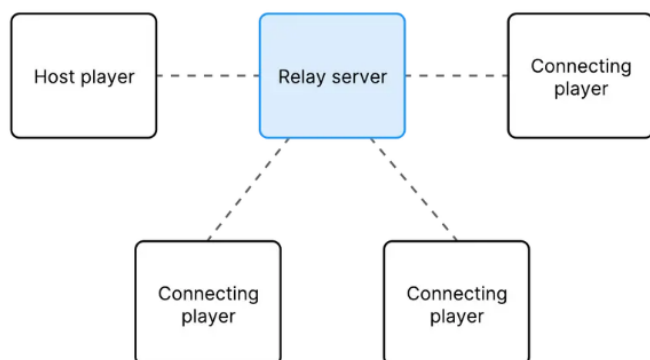
I utgangspunktet er det flere svakheter med dette oppsettet

- Nettverksytelsen påvirkes av å måtte kommunisere med fjernspillere via host-maskinens internettforbindelse.
- Nettverksytelsen kan også reduseres fordi maskinen som kjører serveren potensielt sender mye data. Dette bør få mye oppmerksomhet i utviklingen når vi har en hybrid løsning. Vi må skrive kode der vi er bevisst på hvem som skal synkronisere hvilke data.
- Host har tilgang til all informasjon, noe som gjør det enklere å jukse for spilleren.
- Serveren vil opphøre å eksistere når host forlater spillet.

Vi klarer ikke å fjerne alle negative effekter av å bruke denne arkitekturen, men Unity services tilbyr en løsning som bøter på noen av ulempene: Relay. Vi anser det som et bra kompromiss å introdusere Relay.

3.3 RELAY

Relay er et lav-terskel tilbud som fasiliterer oppkobling mellom spillere uten at utvikleren er nødt til å forholde seg til en dedikert server eller komplikasjoner med peer-to-peer forbindelser. Uten Relay er det nødvendig å sette opp port forwarding med manuell håndtering av IP-adresser, samt å ta ansvar for konfigurering av brannmur. Relay håndterer dette i bakgrunnen. Dette gjør at utviklerne kan fokusere på spill-logikk istedenfor for å forholde seg til NAT og rutere.



Figur 6 [https://docs.unity.com/relay/en/manual/relay-servers\[13\]](https://docs.unity.com/relay/en/manual/relay-servers[13])

Relay er kompatibel med en rekke protokoller (UDP, DTLS og WSS). Etter å ha blitt tildelt en Relay Server, kan klienten kommunisere direkte med Relay ved å bruke en av disse protokollene.

Forespørsler til Relay blir et abstraksjonslag på toppen av den valgte protokollen.

Andre fordeler:

- Sikkerhetstiltak for å beskytte spill-data og forhindre juksing. Kryptering og autentisering for å sikre kommunikasjonen mellom klienter og servere. I tillegg er infrastrukturen utformet for å være pålitelig, med redundans- og feilhåndteringsmekanismer for å minimere nedetid. Herunder smart ruting, trafikkompresjon og adaptiv nettverkstilpasning for å levere effektiv overføring. Denne optimaliseringen bidrar til å minimere forsinkelser og nettverksproblemer.

- Skalerbar infrastruktur. Automatisk tildeling av servere og balansering, slik at spill-økter kan håndtere økende antall spillere. Denne skalerbarhetsfunksjonen er spesielt nyttig for spill med et stort antall spillere eller spill som opplever plutselige økninger i aktive spillere.
- Relay er utformet for å fungere sømløst på flere plattformer og enheter. Relay er kompatibel med ulike operativsystemer, inkludert Windows, macOS, Linux, Android, iOS og konsoller som Xbox og PlayStation.

Alt dette er positivt, men som nevnt klarer vi ikke å unngå alle negative effekter.

Relay abstraherer bort noen nettverkskompleksiteter og utviklerne har begrenset kontroll og innsikt hva som skjer i bakgrunnen. Det gjør at utvikleren potensielt mister en av mulighetene til optimalisering. Det krever for øvrig at utvikleren er kompetent på lavnivå nettverkskommunikasjon. På tross av dette krever Relay fortsatt at utviklerne bidrar med kode for å kommunisere med Relay og andre Unity-tjenester vi benyttet oss av.

3.4 FORBINDELSEN SPILL-DESIGN OG NETTVERKSARKITEKTUR

Vi har tidligere vært inne på at spill-designet avgjør hvilke nettverksløsninger som er aktuelle. Det er ikke en god ide å bruke Relay dersom reaksjonstid er avgjørende for utfallet av en handling. Det er ikke i spillerens interesse at nettverksløsningen påvirker utfallet.

Dette gjelder særlig ressurskrevende spill med intensiv konkurranse. I en ren *Player vs. Player (PVP)* kontekst er det mer naturlig å velge en dedikert server (*DGS*) der alle spillerne er klienter. I vår hybrid-tilnærming er forsinkelsen til host er mye mindre sammenlignet med øvrige klienter [14]. Host har derfor kjempefordel i og med at den fungerer både som klient og server. Det er mye som tyder på at spilltype betyr mye for valg av arkitektur [10]

Vi skal nå forsøke å forsvare vårt valg av nettverksarkitektur basert på hva planen var og hvordan vi designet spillet med hensyn til arkitekturen.

En hybrid-topologi med host er mindre kritisk dersom spillet er strategisk innrettet og basert på samarbeid. Spillerne befinner seg ikke i en spiller mot spiller setting (*PVP*), men spillere mot miljø setting (*PVE*). Reaksjonstiden til *NPCs* som spillerne kjemper mot er fastsatt.

Spillet kan likevel inneholde konkurrerende elementer, men uten at konkurransen er sentral. Konkurransen i vårt tilfelle er ikke direkte mellom spillerne, men forsøkt basert eksterne faktorer. Eksempelvis hukommelse og strategiske valg.

Formålet med spillet er å utforske et komplekst miljø, finne et objekt og returnere med objektet. Gjerne først, fordi da blir man utpekt til vinner (konkurrerende element). I vårt spesifikke tilfelle er objektet en ring er gjemt inne i en mørk labyrint (gruve) og spilleren har begrenset sikt. Avataren er kun omringet av en liten sirkel med lys. Det hjelper ingenting å være først til å finne ringen hvis man sliter med å huske veien tilbake og terger på seg fullt av fiender som blokkerer veien. Å tegne opp labyrinten manuelt er heller ingen god ide for da vil spilleren tape verdifull tid ovenfor andre spillere. En spiller kan også vinne tid ved å plukke opp *power-ups* som øker farten til avataren.

Vi hadde planlagt et poengsystem for utføre heltmodige handlinger; nedkjemping av fiender og å hjelpe andre spillere opp. Incentivet for å gjenopplive andre skulle være poeng-uttelling og fordi det er vanskelig å klare seg alene omgitt av fiender. Det skulle oppleves som et valg. Det er selvsagt mulig å ta sjansen på å kjøre solo og øke risikoen for å ende opp død, men med den fordel at man vinner tid på andre spillere.

Dette var et eksempel på hvordan vårt design påvirket valg av arkitektur.

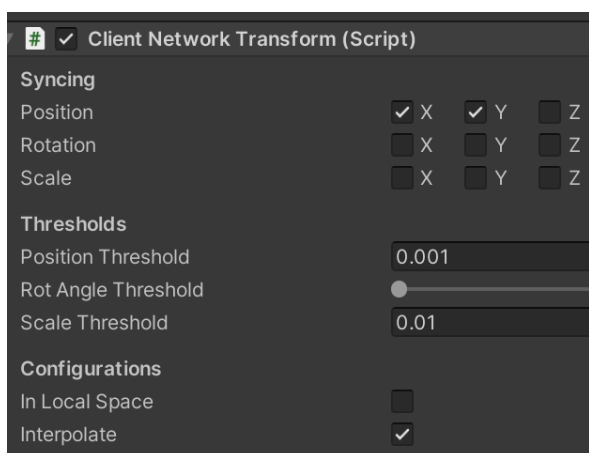
3.5 NETCODE OG LOGIKK

For å implementere nettverkslogikken benyttet vi det relativt nye biblioteket Netcode for GameObjects (v. 1.2.0). I utgangspunktet er all kommunikasjon i NetCode server-autoritativ. Server-autoritativ betyr at enhver forandring hos en klient blir sendt til server før den distribuerer meldingen til alle klienter. Alle avgjørelser blir tatt sentralt og det er bare server som kan modifisere *game-state*. All innkommende data blir sjekket av server før distribusjon. Klienten sender spørsler til server om få lov til å kjøre kode eller sende data, men dette må spesifiseres enten ved bruk av RPCs eller NetworkVariables, som vi kommer mer tilbake til.

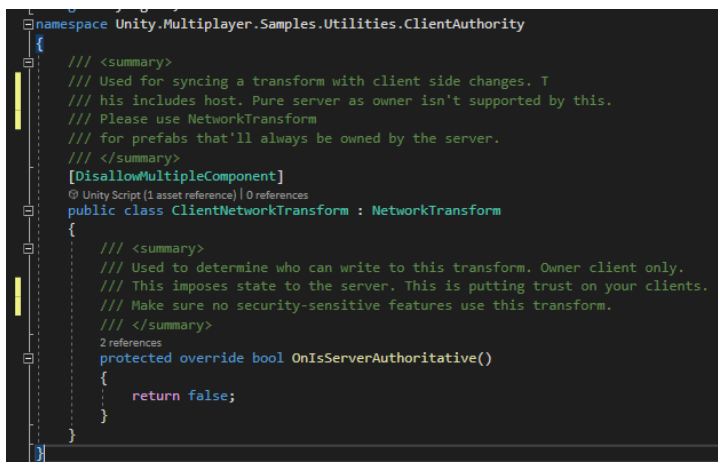
I vår kode arver klassene som skal styre synkronisering fra NetworkBehaviour. Det er basisklassen i Netcode. Når en klasse arver fra NetworkBehaviour blir koden i klassen lest av alle som er tilkoblet. For å hindre unødvendig kjøring av kode og skape forsinkelser er det nyttig å adgangs-begrense hvem som kjører koden. I NetworkBehaviour har vi diverse properties for å bestemme hvilken aktør som har adgang til å eksekvere kode. Dette inkluderer *IsClient*, *IsHost*, *IsLocalPlayer*, *IsOwnedByServer*, *IsOwner*, *IsServer* og *IsSpawned*.

Vi omtalte server-autoritet og klient-autoritet i forrige kapittel. Det eneste som klienten styrer selv i vårt spill er input til avatarens bevegelser. Den åpenbare årsaken til å la klienter sende data uten kontroll er hastighet.

Vi knyttet *Client Network Transform* til avataren/spiller-objektet(bilde 3).



Bilde 3 «Client Network Transform» Unity komponent.



Bilde 4 «Client Network Transform» C# skript.

I vårt tilfelle er det maksimalt fire spillere tilkoblet host. Vi anser det lite sannsynlig at fire ukjente spillere avtaler å spille sammen og jukse. Det mest naturlige er at fire venner spiller sammen for litt vennskapelig konkurranse. Spillerne har dermed muligheten til å utøve en viss indrejustis dersom en av spillerne jukser gjennom å manipulere kode. Vi erkjenner at dette er basert på antagelse. Ikke undersøkelser.

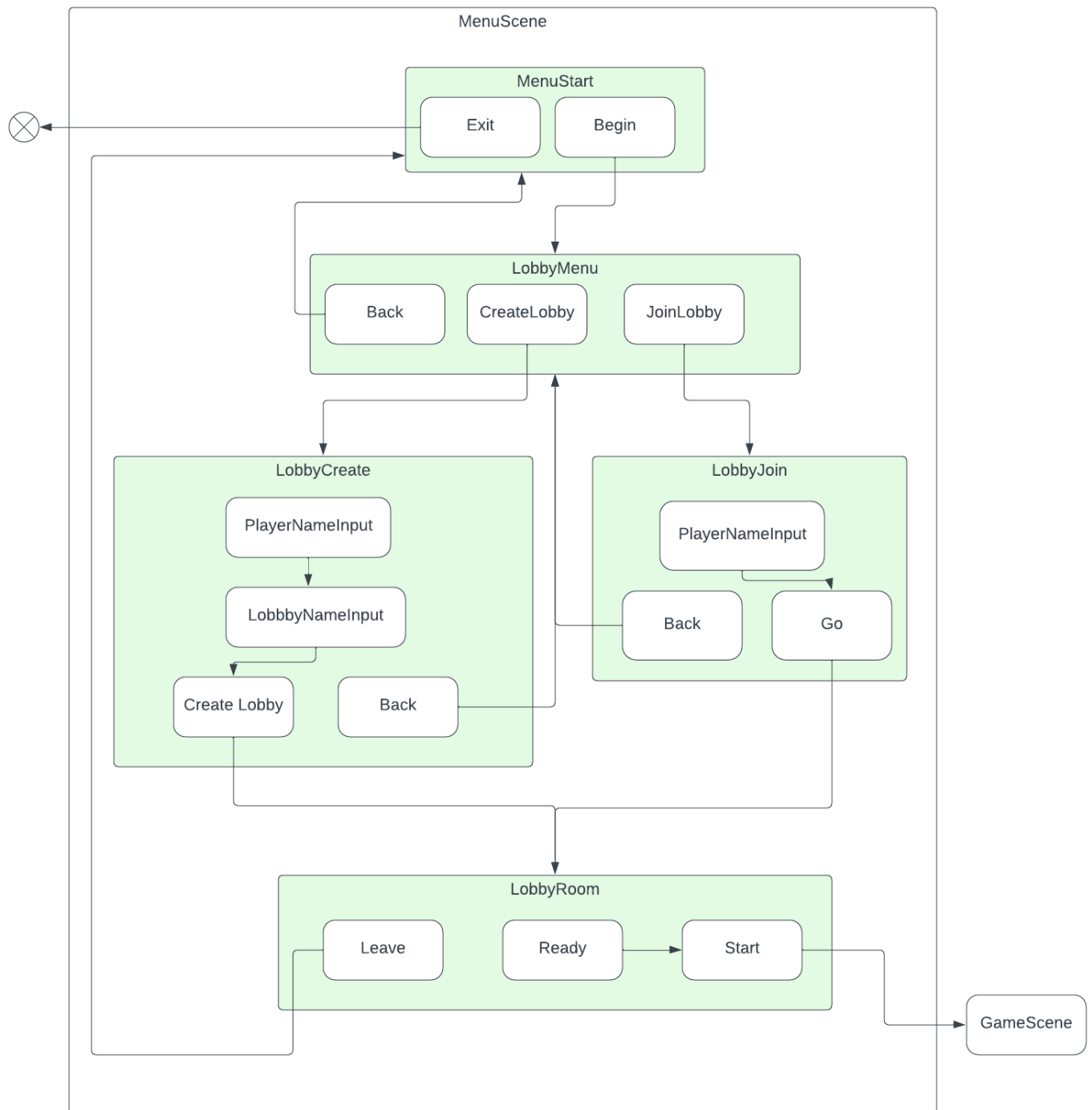
Beskyttelse fra hacking av eksterne aktører er håndtert av Relay. Relay er en fordel med tanke på sikkerhet, siden uerfarne utviklere slipper å sette opp server i et isolert miljø; dedikert server.

3.6 MENYSYSTEM

Spillet starter som de fleste andre spill, med en meny. Menyen er bygget opp av et menysystem som vi har utviklet selv. Menysystemet fungerer ved at man aktiverer og deaktiverer objektene som har UI-elementer i seg.

Tabell 1 Oversikt over menysider

Menyside	Beskrivelse
MenuStart	Den første siden som vises når man starter spillet.
LobbyMenu	Siden tar for seg om man vil fortsette med å opprette en ny lobby eller om man vil koble på en eksisterende lobby.
LobbyCreate	Denne siden har funksjon for oppretting av lobby. Det er inputfelt for spillernavn og lobbynavn.
LobbyJoin	Her kan man koble til eksisterende lobby. Man skriver inn navn på spiller og trykker 'Go' for prøve å koble seg til en lobby og komme seg videre til LobbyRoom.
LobbyRoom	Denne siden tar for seg selve lobbyen hvor spillere samles før de går inn i spillet. I menyen opprettes et chatvindu. Funksjonaliteten til lobbyen blir beskrevet i detaljer senere i denne teksten.



Figur 7 Diagram over flyt i meny

3.6.1 Reaktiv design

I og med at vi utvikler for mobil må vi regne med at spillet skal kunne vises på skjermer med alle mulige typer oppløsning og høyde-bredde-forhold. Unity har noe innebygget funksjonalitet for reaktivt design, men vi fikk det ikke til å fungere tilfredsstillende da elementene i menyen endret seg etter skjermstørrelse, men layout-en ikke forble som vi ønsket. Vi opprettet derfor klassen **MenuLayoutManager** som tar seg av reaktiv skalering. Klassen henter informasjon om høyde og bredde på skjermen spillet befinner seg på. Den beregner deretter størrelser på panel og UI-elementer ut ifra dette.

3.6.2 Tilgjengelighet

Vi har sørget for store knapper med tydelig og beskrivende tekst. Knappene skifter farge når de trykkes på, noe som gjør det enkelt for spilleren å vite om knappetrykket har blitt registrert. Inputfelt har placeholder tekst som gjør at brukeren vet hva som skal skrives inn. Vi har i tillegg varsler for når input inneholder for mange eller for få bokstaver.

3.7 LOBBYSYSTEM

3.7.1 Lobbysystem

Lobbysystemet vår er sentrert rundt Unity sin LobbyService. LobbyService kan brukes til å samle spillere, administrere ventende spillere og gi nødvendige data før et spill startes. RelayService som vi har beskrevet tidligere kan også benyttes sammen med LobbyService for å gi en mer fullstendig løsning for nettverksspill.

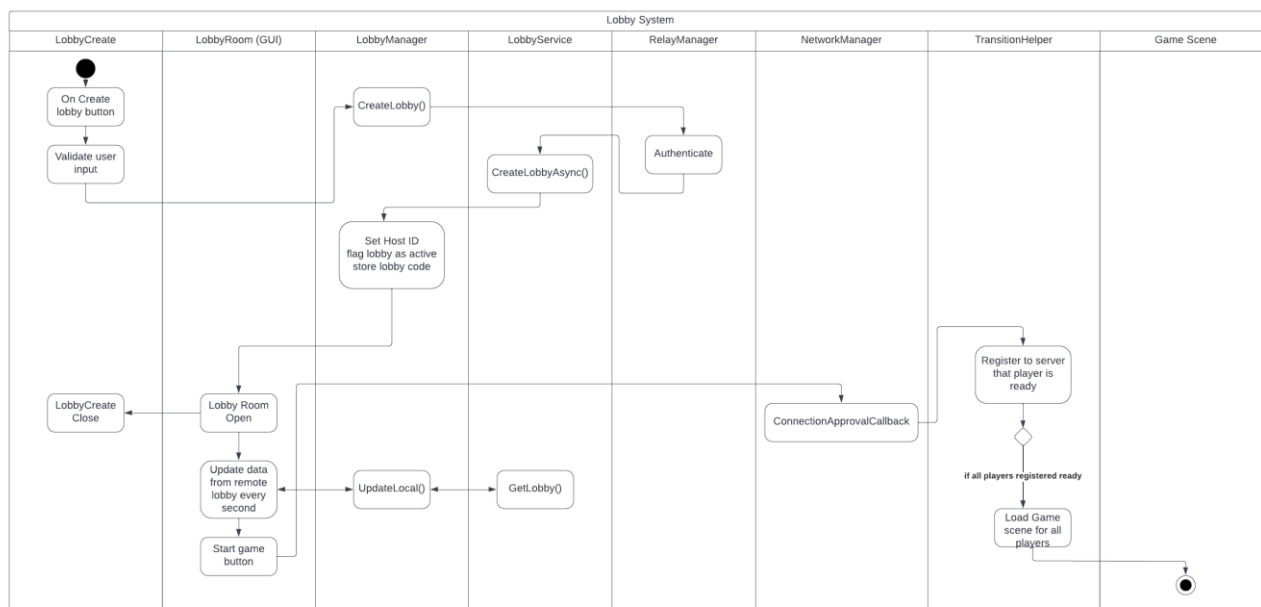
Hvis man bare trenger å håndtere dataoverføring mellom spillere og ikke trenger de ekstra funksjonene til LobbyService, kan man velge å bare bruke RelayService.

Vi startet med å kun bruke Relay, men ønsket etter hvert at spillere skulle kunne koble seg til hverandre uten å måtte dele nøkkelkoder med hverandre. LobbyService har en metode ved navn QuickJoin som tillater dette.

3.7.2 Oppkobling mot LobbyService

Oppkobling mot LobbyService API'et gjøres gjennom vår egenutviklede klasse, LobbyManager. Tilkoblingen mot LobbyService gjøres gjennom API'ets metode, CreateLobbyAsync. Vi bekrefter at forbindelsen til LobbyService er opprettet ved å sørge for at resultatet av denne metoden ikke er av typen null.

API'ets metode CreateLobbyAsync tar inn et argument av typen CreateLobbyOptions. Dette objektet holder informasjon om spillerrelaterte data, uttrykt gjennom et Player-objekt. Player-objektet bærer med seg nøkkelinformasjon om spilleren, inkludert spillerens ID, navn, og en statusindikator for om spilleren er klar til å starte spillet. Dermed gir dette en detaljert representasjon av spillerens identitet og deres nåværende status i lobbyen.



Figur 8 Diagram for flyt i lobbysystem

Diagrammet i figur 9 viser vårt egetutviklede lobbysystem, delt inn i swimlanes. Hver swimlane representerer en klasse som igjen har metoder i seg. Pilene mellom metodene representerer flyten i lobbyen.

Kort fortalt ser vi at vi starter i menyen LobbyCreate hvor brukeren oppretter en lobby. Vi går deretter videre til LobbyManager som tar seg av tilkoblingen til LobbyService og RelayManager. Videre åpnes den lokale lobbyen og LobbyRoom starter å oppdatere informasjon basert på hva den får fra LobbyManager. Når lobbyens host trykker på start game, går flyten videre til Networkmanager som gir en approval callback. Videre opprettes en transitionhelper som tar seg av synkroniseringen av overgangen til neste scene.

3.7.3 Lokal lobby

Klassen LobbyRoom tar for seg den lokale lobbyen som opprettes når en spiller får tilgang til en lobby. Den lokale lobbyen får informasjon fra den eksterne lobbyen gjennom polling updates. Spillerens ready-status og navn oppdateres for hver polling update.

3.8 SYNKRONISERING

I et multiplayer online spill er det nyttig å ha oversikt over hvilke data og operasjoner som skal synkroniseres. Da er vi i stand til å ta designmessige grep i favør av at mest mulig av spillet kjøres lokalt, slik nettverkstrafikken holdes på et minimum.

Vi har tidligere nevnt at nettverksklasser arver fra NetworkBehaviour. En abstrakt klasse med hensikt å generere unik nettverkslogikk gjennom at utvikleren bruker funksjonalitet i NetCode. Netcode er et bibliotek for alt relatert til nettverkskommunikasjon i Unity. Alle GameObjects som skal synkroniseres med NetworkBehaviour krever at vi kobler NetworkObject komponenten til GameObjects.

Vi har tre måter å synkronisere på. RPCs (Remote Procedure Calls), tilpassede meldinger (Custom Messages) og NetworkVariables. Samtlige blir brukt til å synkronisere tilstander og hendelser, men individuelt har de hvert sitt bruksområde avhengig av kontekst.

RPCs er inndelt i ServerRpc og ClientRpc. ServerRpc gir klientene mulighet til å sende meldinger til serveren. Det vil si at informasjonen i meldingene blir behandlet på serveren. Motsatt er ClientRpc en metode for serveren til distribuere meldinger ut til klienter, men også spesifikke klienter [15]. RPCs har restriksjoner på hvilke typer det er lov å sende. Det må være primitive non-nullable typer, med unntak av string.

Tilpassede meldinger eller Custom Messages gir muligheten til å designe egne meldinger uten bruk av NetCode. Disse meldingstypene er ikke tilknyttet et GameObject. Formålet er å bygge et eget meldingsutvekslingssystem, der man selv bestemmer hvilke typer det er lov å synkronisere. Vi har ikke brukt denne tilnærmingen, og vi går derfor ikke nærmere inn på detaljene.

NetworkVariables er den tredje muligheten for å synkronisere. Man setter bruker-rettigheter i opprettelsen av nettverksvariabelen (bilde 5).

```
NetworkVariable<int> netvar = new(readPerm: NetworkVariableReadPermission. Everyone, writePerm: NetworkVariableWritePermission. Owner,  Server);
```

Bilde 5 Alle eller bare eieren av nettverksobjektet kan lese. Kun eier eller server kan skrive. Owner vil si den brukeren som kontrollerer GameObject'et med NetworkObject komponent.

Nettverksvariabler støtter primitive typer, Unity unmanaged built-in types og enums. Det er også mulig å bruke egendefinerte typer, som brukeropprettede klasser, men det krever at man implementerer

INetworkSerializable, slik at man får serialisert objektene man ønsker å sende. Hvis man ønsker å sende streng-objekter må de være av en forhåndsbestemt størrelse. Det finnes flere utgaver av `FixedString[ant. Bytes]` til dette formålet som hver for seg sender et bestemt antall bytes. Dette har vi brukt i navnene over hodet til avatarene (*Heads Up Display/HUD*), slik at de kan se navn til hverandre.

`NetworkVariables` er egnet til å ivareta hendelser som skal vedvare. Som at en dør skal forbli åpen i en tidsperiode. RPC kall er enklere og raskere, og er aktuelt ved momentane forbigående hendelser(transient) F.eks. animasjon av en eksplosjon. Det er ikke kritisk om en spiller som joiner midt i et pågående spill ikke får sett en animasjonen (RPC), men det ville bryte med spill-logikken om døra ikke holdt seg åpen (`NetworkVariable`) når den er åpen hos andre klienter.

Vi skal nå presentere `GameObjects` og tilhørende funksjonalitet vi har synkronisert. Alle skript arver fra `NetworkBehaviour`. Vi starter med spiller-objektet eller avataren som er kontrollert av spilleren.

3.8.1 Player

Vi valgte å gjøre transformasjonene til avataren klientautoritativ slik at spilleren kontrollerer avataren lokalt. Hvis vi hadde valgt en server-autoritativ tilnærming måtte input først ha blitt sendt til server og tilbake. I et slikt tilfelle måtte vi også vurdert å implementere «player prediction» som nevnt i kapittel 2.3.1. Dette regnes for å være krevende.

Isteden får eieren av objektet rask respons på input fordi de slipper å gå via server først. Videre betyr det at vi stoler på at spilleren oppfører seg redelig og ikke jukser. Det følger av dette at animasjon hos avataren er *owner authoritative*. Eieren av klienten oppdaterer sine egne animasjoner lokalt.

3.8.2 Helse til avatar

Visualiseringen av Helsen til Player er bygd opp av RPCs og C# events. Vi har et skille mellom utregning av helse og visualisering av helse. Vi er ikke interessert i å synkronisere selve endringen i helsen til spilleren. Isteden bruker vi utregningen til å synkronisere status på healthbar UI til avataren.

Først sender klienten en `ServerRPC` om at helsen har blitt forandret. Serveren lager en `ClientRPC` som gir beskjed om at visualiseringen av helse hos avsenderen skal oppdateres hos klientene.

Når det kommer til synkronisering av at en avatar er død må vi i tillegg bytte ut animasjonen hos klientene. En avatar som er på 0 i helse har annen grafikk enn avataren som fortsatt er live. Da er det nærliggende å bruke event i tillegg til RPCs. Klienten som eier avataren som er på 0 liv forespør serveren om å visualisere hendelsen. Serveren distribuerer meldingen om at en spiller er død ved at den kaller opp hendelsen(event) i en `ClientRPC`. De som abonnerer på hendelsen vil se at animasjonen skifter status fra levende til død.

Player (objektet) abonnerer på hendelsen. Det innebærer at vi kan utføre operasjoner på spilleren som er død. Det opplagte tilfellet er å frata spilleren kontrollene slik at han ikke kan bevege sin døde avatar.

Det er også powerups som gir økt helse. Disse benytter de samme metodene for endring av helse som når en spiller mister helse. Endringene er synkronisert for alle klienter.

3.8.3 ChatManager

Når en spiller skriver i chatten og trykker Enter vil serveren kjøre en `ClientRPC` med meldingen, avsender navn og klient ID. Når det skjer, legger vi navn og melding i en distribuert liste på følgende format

```
_messages.Add($"{senderName}: {message}"); //distribuert
```

Der `_messages` er lista. Dette er altså en distribuert liste. Lista har en begrensning på 10 beskjeder for å ikke belaste nettverket unødvendig. For hver melding som overstiger dette antallet vil den første bli fjernet fra lista. First in First out (fifo). Men output til chatvinduet blir konstruert lokalt. I Update

metoden, som kjører hver frame, blir teksten fortløpende satt sammen. Output til chatvinduet er en sammenhengende tekst bygd opp av innholdet fra `_messages`.

```
//lokalt
foreach (string s in _messages)
{
    newContent += s + "\n";
}

chatContent.text = newContent; //vis i tekstboks
```

Dette er på en måte motsatt av hva vi gjorde i skriptet som behandler visualisering av helse til avataren. Her er det lista med beskjeder som blir synkronisert. Det visuelle håndteres lokalt.

3.8.4 NPCs

Enemy er vår *NPC*. Den opererer på egen hånd. Den er styrt av server i sin helhet og dermed server-autoritativ. På samme måte som Player, har vi utelatt de transformasjonene som ikke gir mening i spillet. Det er kun forflytning i x og y.

I likehet med Player abonnerer Enemy på PlayerHealth slik at den kan vite når avataren den angriper er død. Når denne hendelsen inntreffer kaller vi `StopAnimationClientRpc` og setter tilstanden på fienden til `State.PlayerDown`. Animasjonen stopper hos klientene, og fienden står stille i noen sekunder før den går tilbake til `State.Roaming`.

3.8.5 Lys

Alle objekter som befinner seg i gruen har et materiale knyttet til seg som gjør at de kun vises når de får lys på seg. Lys opprettes som et objekt i unity og kan plasseres hvor som helst i scenen.

Det er ikke noe frittstående lys i gruen. I stedet har vi et lys knyttet til hver avatar. Lyset aktiveres når avataren ankommer gruen, men det er ikke tilstrekkelig å kun aktivere lyset lokalt. For at spillere skal se hverandres lys er det nødvendig at vi synkroniserer aktiveringen av lyset over nettet.

Synkronisering av lys har vi valgt å gjøre gjennom Remote Procedure Call(RPC)-metoder. La oss betegne en spiller som ankommer gruen for spiller A. Spiller A kaller da på en `serverRPC`-metode. Serveren svarer på denne metoden ved å kalle på en `clientRPC`-metode som kjøres av alle tilknyttede klienter. I metoden følger det et argument med spiller A sin `'networkObjectId'`. Dette er en unik id som `NetworkManager` har registrert. Med denne kan hver klient finne spiller A i sin lokale instans og aktivere lyset for denne spilleren.

Powerups for fakkel fungerer på samme måte. Når spiller A plukker opp en fakkel vil han sende ut et kall på en `serverRPC` metode som igjen kaller på en `clientRPC`-metode. `ClientRPC` metoden kjører for alle tilknyttede klienter og finner spiller A ved hjelp av `'networkObjectId'`. I denne metoden vil spiller A sin lysradius økes. Denne metoden har også en timer knyttet til seg som kjører i en `coroutine`. Når timeren har telt ned vil fakkelen radius settes til sin opprinnelige størrelse.

3.8.6 Speed powerup

Når en spiller kolliderer med en speed powerup vil avatarens fart økes i en viss tid før den settes tilbake til opprinnelig verdi. Det er ikke nødvendig med ytterligere synkronisering for å få dette til. Synkronisering av avatarers bevegelse allerede er implementert.

3.9 MER OM ENEMY

Enemy har tre aktiviteter gjennom en *statemachine* eller tilstandsmaskin: `Roaming`, `Playerdown` og `ChaseTarget`. I `Roaming` vandrer den omkring og tar konstant avgjørelser om hvilken retning den skal

gå. Den bruker et tilfeldig antall sekunder i intervallet [1, 4] for å bestemme seg. Retningen er også et tilfeldig valg mellom `Vector2.right`, `Vector2.left`, `Vector2.down` og `Vector2.up`

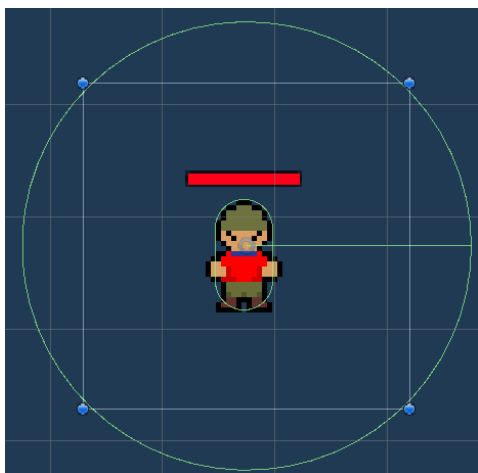
De tre statusene kjører konstant i *FixedUpdate* metoden og bytter når de får beskjed. *FixedUpdate* betyr fast oppdateringsfrekvens; like lenge mellom hver gang metoden kjører. Dette er fordi vi vil at det er fysikkegenskapene som skal bestemme hvordan animasjonen ser ut. *FixedUpdate* er en override-metode i likhet med *Update*. I *Update* vil derimot oppdateringsfrekvensen være opp til hvor fort maskinen klarer å kjøre og dermed varierer oppdateringsfrekvensen. I *Update* kjører vi fiende angrep, men kontrollerer at *Enemy* ikke angriper oftere enn 1 gang pr. sekund.

Det er to måter for *Enemy* å forflytte seg på. Den ene i *Status.Roaming* der den først velger en tilfeldig retning i form av en retningsvektor. Deretter flytter vi i den retningen ved å kalle *Translate* på *Enemy*. *Translate* er en vektor. Vi sender inn retningen og ganger med valgt hastighet og en tidsverdi. Tidsverdien *Time.deltaTime* er tiden det har gått siden forrige frame(bilde 6).

```
moveDirection = moveDirections[currentMoveDirection];  
transform.Translate(SPEED_VALUE * Time.deltaTime * moveDirection);
```

Bilde 6 Viser hvordan fienden vandrer rundt når den er i tilstanden *Status.Roaming*

Enemy bruker en *Circle Collider* med en *trigger* aktivert. Vi bruker *trigger* til å detektere *Collidere* som tilhører avatare. *Circle Collider* har en rekkevidde 2,5 ganger høyden av *Enemy*-objektet.



Bilde 7 *Enemy* i Unity editor med to *Collidere*. *Circle Collider* ytterst. *Capsule Collider* innerst, formet som en kapsel.

Vi bruker *Circle Collider*(bilde 7) til å skape en illusjon av at *Enemy* velger å forfølge den avataren som er nærmest. Å forfølge betyr at avataren blir satt som mål eller target. Når en avatar bryter grensen til *Circle Collider* vil *Enemy* velge denne avataren som target. Dette gjelder uansett om *Enemy* allerede har en target. Derfor vil *Enemy* til enhver tid bytte target til den siste avataren som kolliderte med *Circle Collider*.

Enemy har i tillegg en *Capsule Collider* med *trigger* for å se om en avataren er nær nok til at *Enemy* kan angripe. Collideren ligger tett inntil *Enemy* slik at avataren kan komme nært nok til å angripe uten å bli angrepet selv.

Når en avatar kommer nær nok spiller angreps-animasjonen og fienden svinger sverdet i retning avataren.



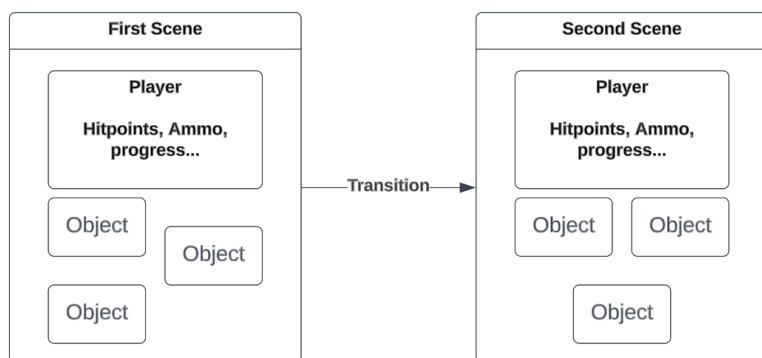
Bilde 8 Fienden svinger sverdet i respons til avataren som er nær

Animasjonen aktiverer et spillobjekt tilknyttet en *Polygon Collider*(bilde 8). Den er utformet slik at den dekker hele området sverdet svinger i.

Angrep fra *Enemy* har en begrensing. Ett angrep per sekund. Når en spiller blir truffet av *Polygon Collider*en kjører programmet et skript som sjekker hvem eieren til sverdet er. Hvis *Enemy* eier sverdet og den treffer avataren kaller vi en metode *SwordCollision* der avataren får skade.

3.10 BYTTE AV SCENER OG SCRIPTABLE OBJECTS

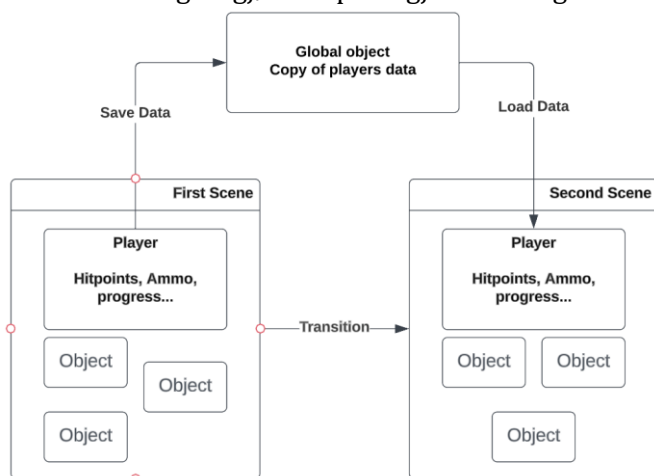
Figur 9 og 10 beskriver problematikken ved skifte av scener i Unity. Mellom hver scene blir alt innhold tilbakestilt til opprinnelig status. Nettverksobjekter rekonstrueres mellom hvert sceneskift, og verdier i tilknyttede script blir borte. Hvordan skal unngå å miste data?



Figur 9 Basert på bilder fra Sitepoint.com [16]

Vi kunne brukt lagring til fil eller database, men ikke noe man vanligvis bruker i bytte mellom scener. Database-oppsett blir for omfattende og tar for mye båndbredde i forhold til størrelsen på prosjektet. Vi har bare noen få GameObjects som spawnes på nettverket.

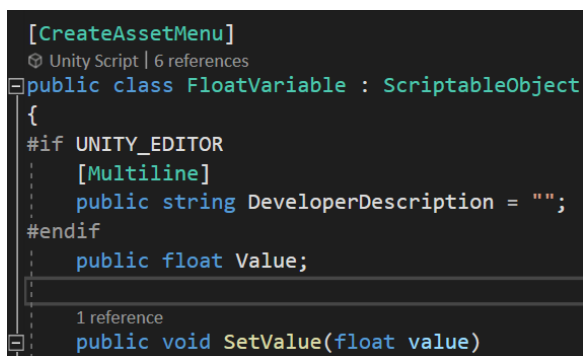
Måten vi vanligvis gjør det på er gjennom et globalt GameManager objekt, som inneholder kopier.



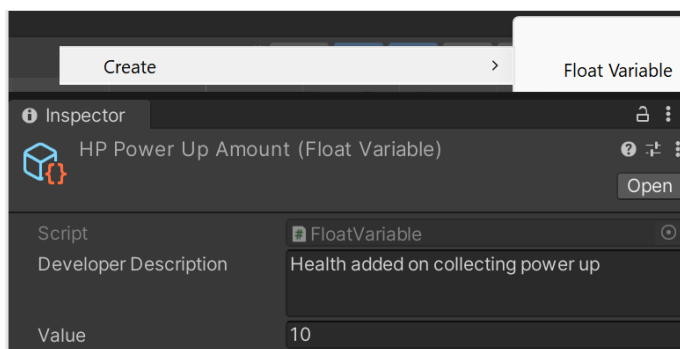
Figur 10 Basert på bilder fra Sitepoint.com [16]

GlobalObject er en singleton som vi kaller DontDestroyOnLoad på, samtidig som vi sjekker at det ikke er noen forsøk på å lage kopier av denne klassen.

Alternativet er Scriptable objects. For å opprette scriptable objects trenger vi en klasse som arver fra ScriptableObject(bilde 9). Koden vi fyller klassen med er ikke syntaktisk forskjellig fra annen kode, men innholdet skrives til en asset som oppbevares på disk(bilde 10). Dermed vil forandringer vi gjør i en variabel i et scriptable objekt vedvare etter at programmet er avsluttet.



Bilde 9 Dekorererer med CreateAssetMenu som gjør at vi får opp en meny i Editoren der vi kan opprette objektet.



Bilde 10 Nytt menyvalg->Float Variable. Da får vi objektet (visualisert som en variabel) vist ovenfor. Verdien lagres permanent i Asset Mappen

3.10.1 Scriptable objects

Vi brukte scriptable objects til å ta vare på verdier i overgangen mellom Meny/Lobby scene og Game scene. Navnene til spillerne ble opprettet i hvert sitt scriptable object som ble lagt i en liste. I tillegg har vi et scriptable object med gamestatus som tar vare på informasjon om spillet er startet, hvilken plattform spillet er startet på og posisjon for dør i uteområde.

3.10.2 LoadingManager

Før spillet starter, er alle tilkoblede klienter i en felles lobby i Menu scenen. Fra denne lobbyen kalles det på en sceneovergangsmetode som formidler overgangen til spillets hovedscene. Selv om alle klienter instrueres til å bytte scene samtidig, er det nettverksforbindelsen som dikterer hvor raskt hver spiller vil bli plassert inn i hovedscenen. Hvis en spiller klarer å starte før de andre, vil dette potensielt kunne påvirke balansen i konkurransen negativt.

For å løse dette problemet, har vi designet en egen klasse, LoadingManager. Denne klassen håndterer synkroniseringen av overgangen til hovedscenen. Med denne klassen sikrer vi at alle spillere har fullført innlasting av scenen og alle nettverksobjekter, før de får tillatelse til å styre sin avatar. Vi bruker nettverksvariabler for å håndtere denne synkroniseringen.

LoadingManager inneholder en metode designet for å vente på at alle klienter skal koble seg til spillet. I mellomtiden vises en lasteskjerm for alle klienter. Når server registrerer at alle klienter har lastet inn alt innhold, endrer serveren statusen til nettverksvariabelen 'networkedClientsReady' til 'true'.

Nettverksvariablene har fra netcode en event .OnValueChanged. Denne hendelsen utløses når det er en endring i verdiene til variablene. Dette gir hver enkelt klient muligheten til å reagere på endringene som serveren initierer i denne spesifikke variabelen.

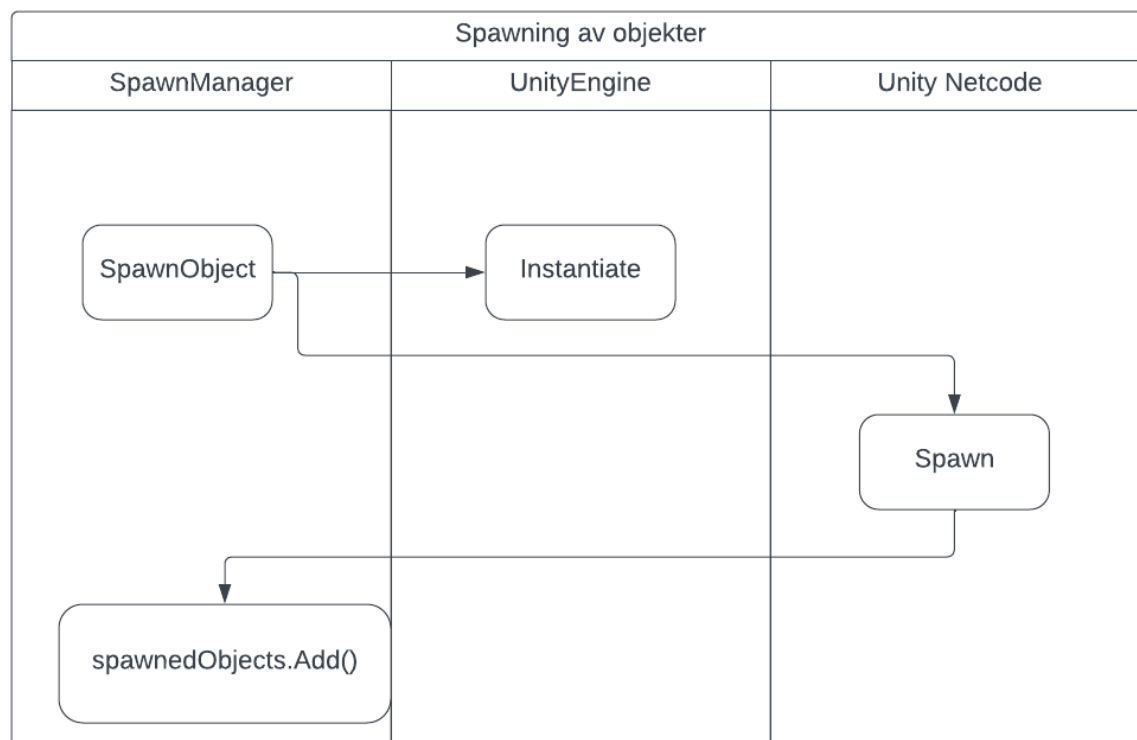
Med denne endringen settes det i gang en nedtellingsmekanisme på hver klient. Når denne nedtellingen er ferdig på serveren, aktiveres hendelsen 'networkedCountdownFinished'. Dette betyr at kontrollene for alle spillere er aktivert og klar for bruk, noe som tillater spillerne å begynne å styre deres respektive karakterer.

3.11 KARTSYSTEM

Spillområdet vårt er laget med Unity sine Tilemaps. Tilemaps er et system som deler spillområdet inn i et rutenett hvor vi selv setter inn tiles for å designe området. Vi har utviklet oppå dette systemet for å kunne implementere en tilfeldig plassering av powerups og fiender. Dette systemet befinner seg i klassen TilemapManager. Denne klassen tillater oss å plassere objekter dynamisk. For å gjøre dette, velger TilemapManager først en tilfeldig tile på kartet, før den sjekker om denne tilen er tom. Dette

tillater oss å tilføre en tilfeldig og uforutsigbar dimensjon til spillet, noe som øker spenningen og utfordringen for spillerne.

3.12 SYSTEM FOR SPAWNING AV OBJEKTER OVER NETTVERK



Figur 11 Spawning av objekter over nettverk

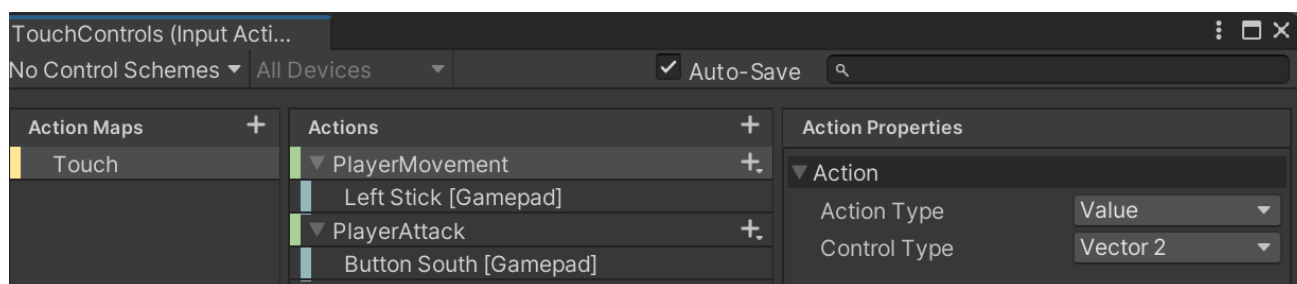
SpawnManager er vår egne manager som tar for seg instansiering av nettverksobjekter. Den har først et kall på UnityEngine sin Instantiate. Deretter gjør den et kall på Netcode sin Spawn metode. Når dette er ferdig vil objektet legges til i en liste over spawnedde objekter.

3.13 ANDROID OG TOUCH-KONTROLL

Et av kravene til prosjektet er kryss-plattform. Med plattform mener vi operativ-system og ikke fysisk enhet. Vi valgte Android fordi markedsandelen er mye større sammenlignet med konkurrenten IOS [17]

I tillegg tok mobilen opp 50% av spillmarkedet i 2022, i forhold til pc som bare tar opp 22%. [18, s. 23]

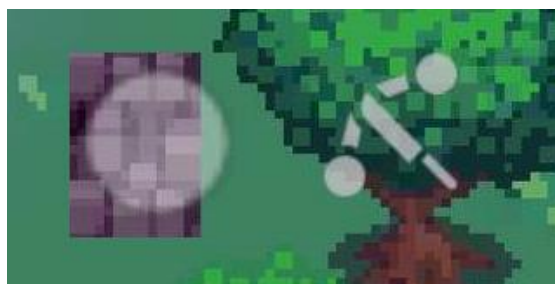
Unity introduserte et nytt Input System i slutten av 2019. Vi brukte den gamle input-manageren inntil vi skulle implementere touch kontrollere for Android. Dokumentasjonen på det nye systemet virket bedre. Vi kunne sette opp relasjonene på det nye systemet i editoren og konsentrere oss om logikken til kontrollene i koden. Vi opprettet en asset av typen 'Input Actions' og kalte den for TouchControls.



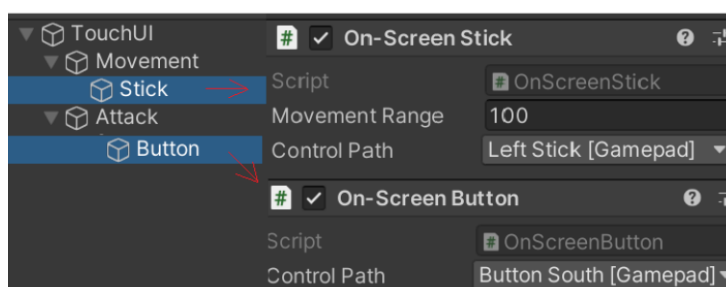
Bilde 11 Forbindelser mellom Actions og egenskaper. PlayerMovement gir en verdi i form av en 2D vektor. Vi ønsker at kontrollene skal simulere oppsettet på en Gamepad.

Etter oppsett av TouchControls trengte vi en 'Player Input' komponent på objektet som spilleren kontrollerer. Vi legger inn en referanse til TouchControls på 'Player Input'. Dermed kunne vi beskrive i koden hvilke hendelser som inntreffer når en Action av type PlayerMovement eller PlayerAttack blir avfyrt.

Den siste biten er visualisering av kontrollene og knytte de opp mot konfigurasjonen i TouchControls. I Unity kan vi gjøre dette i editoren. Vi valgte noen passende symboler fra standard biblioteket (se bilde 12). Bevegelsesretningen til avataren er kontrollert av en digital joystick. Avataren angriper når spilleren trykker på sverd-symbolet.



Bilde 12 Touchcontroller er gjennomskjinnelige slik at de ikke skjuler elementer i spillet



Bilde 13 Knytter touch-symbolene opp mot Gamepad valgt i TouchControls. Joysticken har en radius på 100 pixeler spilleren kan bevege den innenfor

I tillegg måtte Player objektet få en Character Controller. Denne kontrolleren er den komponenten som flytter avataren etter at input er mottatt fra joystick. Uheldigvis kom Character Controller i konflikt med eksisterende regime vi bruker til å flytte avataren i Windows. Løsningen var å ha komponentene til kontrollene på forskjellig nivå hierarkiet (ref. HandleTouchInput() i PlayerBehaviour for kode).

I forhold til kryssplattform har Unity adoptert filosofien «write once, run everywhere» (WORA). Et motto lansert av Sun Microsystems i 1995 i forbindelse med utgivelsen av programmeringsspråket Java[19]. I praksis betyr dette at vi kan bygge prosjektet til den plattformen vi ønsker uten å skrive kode spesifikt for hver plattform separat.

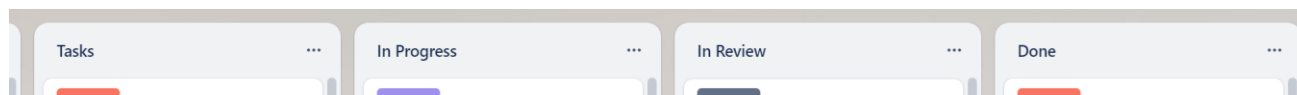
Vi måtte imidlertid ta høyde for forskjellige kontrollregimer på Android og Windows. Plattform blir detektert ved oppstart og initialisert før spillet starter. I koden kjører vi en sjekk og aktiverer kontroller og visualisering i henhold til plattform.

3.14 ARBEIDS OG ROLLEFORDELING

Rollefordeling er en funksjon av størrelsen på teamet og ikke minst økonomiske rammer. Er utvikleren et lite uavhengig team med privat budsjett eller et stort multinasjonalt selskap? I et stort team er det gjerne skarpt skille mellom kreative og tekniske roller. I mindre team, som vi vårt tilfelle, er det mer naturlig å dele på rollene. Likevel tok vi hensyn til kunnskap og erfaring.

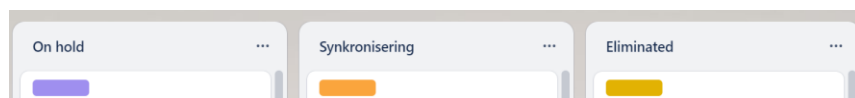
Mikal var den eneste med erfaring fra 2D-grafikk, sprites og animasjon. Tore er kjent med Photoshop og brukte AI generatoren Midjourney som grunnlag for introduksjonsbildet og poster. Han tegnet også storyboard. Carlos designet systemer og kode for blant annet managers, menyer og lobbysystem.

Utover dette var det stort overlapp i ansvar og vi plukket ledige oppgaver fra backlog. Rollefordelingen ble organisert på Trello hvor vi meldte oss på oppgaver. Systemet har et tradisjonelt oppsett inndelt i kolonner. I tasks skrev vi en overordnet utdypning av hver user story (med samme nummer som i User Story) i tillegg til underoppgaver.



Bilde 14 Vi flyttet oppgavene («Tasks») fra venstre mot høyre etthvert som utviklingen gikk fremover. Lengst til venstre er «user stories» (ikke vist på bilde).

Vi innførte også noen ekstra kolonner som ikke var knyttet til fremdrift, men for å hjelpe oss selv å få oversikt. En kolonne for de oppgavene («tasks») som må synkroniseres (over nett), en kolonne for oppgaver vi hadde nedprioritert («on hold») og en kolonne for oppgaver vi hadde eliminert.



Bilde 15 Kolonner med statisk innhold

Alle var aktive i feilsøking. Veilederne hadde også tilgang til Trello og kunne følge utviklingen der.

For en mer detaljert fordeling av ansvarsområder henviser vi til Trello[20]

3.15 GRAFIKK, ANIMASJON OG 2D OVENFRA PERSPEKTIV

Vår oppdragsgiver understreket at det er tilstrekkelig med enkel grafikk. Derfor valgte vi å lage spillet i 2D.

Oppsettet ble valgt fordi det med stor sannsynlighet er enklere å bruke 16-bits tegninger til objekter og bakgrunn, i motsetning til 3D-modeller. 3D-modeller må modelleres i eksternt program dersom vi skal ha humanoide figurer. En 3D-modell som skal animeres krever et skjelett med kinematisk oppsett (*rigging*). Derne st må vi koble skjelettet til grafikken/polygonene på figuren (*skinning*). Import fra et eksternt 3D-program til Unity kan også by på utfordringer. Dette er for tidkrevende i forhold til det oppgaven etterspurte. Selv om en av oss har erfaring med 3D ble alternativet med en 3D-verden raskt avfeid fordi det ikke er relevant for prosjektet.

Animasjonene spiller ti bilder per sekund slik at bevegelsene til avatarer og fiender visualiseres på en tilfredsstillende måte.

Vi har forsøkt å holde visualisering adskilt fra logikk i opprettelse av spillobjekter. Vi ønsket at logikk og visualisering skulle være på forskjellig nivå i hierarkiet. Det er ikke kritisk i et spill av mindre omfang, men det er en god vane. Ideen er å unngå at forandringer i logikken påvirker visualiseringen og motsatt. Et eksempel er koordinater. Koordinatene vil endre seg dersom spillobjektet blir skalert, når visualisering og logikk er på samme nivå.

3.16 SPILLMILJØ OG GAME DESIGN

I forprosjektet og kravspesifikasjonen hadde vi en visjon om hvordan spillverden skulle virke. Ikke alt var krystallklart og det ble flere endringer. Men vi hadde en felles forståelse for retningen spillet skulle

ta. Visjonen ble presentert gjennom game design dokumentet (GDD) og storyboard. Det skulle være et lite spill med en begrensning på fire spillere samtidig.

Bakgrunnshistorien er at spillerne skal finne en tapt ring. Ringen befinner seg i en mørk gruve der spillerne har begrenset sikt. For å finne gruva må spillerne først gjennom et uteområde å lete etter inngangen. Gruveinngangen er plassert på et tilfeldig sted i uteområdet slik at spillerne kan gå sammen for å øke sjansen for å overleve, eller splitte opp i håp om å finne gruveinngangen først og få et forsprang foran andre spillere.

Med dette oppsettet kommer den påkrevde chatten til å bli en mer hjelpsom funksjon i spillet. Spillere kan utveksle meldinger med hverandre om hvordan de kan samarbeide, og fortelle når de finner noe interessant.

Målet er å returnere ringen til eieren på begynnelsen av spillet. Fra begynnelsen av blir spillet en førstemann-til-mølla-konkurranse.

I både uteområdet og gruva er det fiender (*NPCs*) som vandrer rundt for å gjøre letingen farligere. Utfordringen blir å navigere rundt fiender og følge godt med fiendens bevegelser siden de går i tilfeldige retninger.

Fiendene jakter aktivt på avatarene til spillerne. Fiendene har sverd og angriper avatarene. Avatarene har også tresverd og kan skade fiendene, men sverden har beskjeden skade. Avatarene og fiendene har et «heads up display» (HUD) som viser hvor mye liv de har til enhver tid.

Rundt omkring på kartene er det power-ups/potions. En type power-up øker farten noen sekunder. En annen gir tilbake en brøkdel av helsen. Den første som returnerer med ringen vinner spillet.

Hvis spilleren blir oppdaget av fienden eller det er umulig å komme forbi gjelder det å angripe mens de er nær nok til å angripe og holde dem langt nok unna så de ikke kan angripe tilbake. Utfordringen her er å holde riktig avstand fra fienden og ha god presisjon når man angriper [4, s. 262].

De fleste bildene vi bruker når spillet er i gang kommer fra gratis ressurser på Itch.io. Spiller -og fiende-figurene kommer fra Youtube-kanalen til «Heartbeast»[21]. Vi redigerte bildene til å passe vårt tema.

Stilen for det første området i spillet var en skog med gress, steiner, trær og ruiner [22], mens stilen for gruva er et mørkt område med steingulv -og vegger [23].

4 RESULTATER

4.1 MÅLOPPNÅELSE

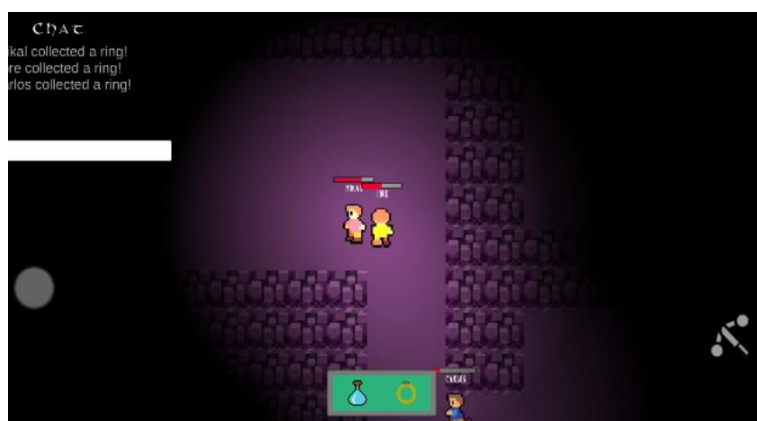
Vår oppfatning er at vi kom i mål med samhandling. Spillerne ser avatarene til hverandre og avatarene kan ikke okkupere samme område. Spillerne står sterkere sammen mot fiender og kan forsvare



Bilde 16 Kjemper mot fienden sammen



Bilde 17 Minimalistisk chat



Bilde 18 Bildet er tatt på mobil. Avatarene blir påvirket av lyset til hverandre. Mer lys hvis flere avatarer bærer fakler og er nær hverandre

hverandre. De kan samarbeide ved å lete etter gruve-inngangen enten separat eller delt. Hvis de leter hver for seg kan spilleren som fant inngangen melde fra i chatten om hvor den befinner seg.

Samtidig må vi ikke overse at målet med spillet er å finne ringen og returnere den. Den som utfører bragden blir utpekt til vinner. Ulempen ved å kjøre solo er som tidligere nevnt at spilleren blir stående ovenfor flere fiender alene og fiendene sikter seg inn kun på den ene avataren. Da må spilleren ta stilling til om det hadde vært bedre å samarbeide. Et *risk-reward* system. Med et poengsystem og med mulighet til å gjenopplive andre hadde spillet angivelig fungert enda bedre.

Grafikken er vi fornøyd med. Den er enkel, men tilstrekkelig slik oppgaven krevde.

Vi har en fungerende chat for å skrive enkle meldinger. Meldingene blir distribuert korrekt. Vi bytter fokus mellom chat og spilling slik at chat-input ikke beveger avataren. I tillegg bruker vi chatten til å sende ut noen meldinger fra *NPC-wizard*-karakteren som delte ut oppdraget i begynnelsen. Dette er for å gi litt ekstra stemning og atmosfære.

I gruveområdet fikk vi implementert et system for lys. Spilleren ser ikke mye fordi avataren er omgitt av en liten sirkel med lys. Sirkelen kan bli større ved at avataren plukker opp fakler. Systemet er synkronisert og avatarene blir påvirket av lyset til hverandre.

Avatarene har liv og fiendene har liv. Dette gjør at spillerne kan ta taktiske beslutninger. For eksempel kan de med mest liv forsvare avataren med minst liv. Vi har *power-ups* som gjør at avataren gjenoppretter en porsjon med liv. Dermed kan avataren med minst liv prioriteres

når det gjelder hvem som skal få liv-*power-up*. *Power-up* forsvinner når en avataren kolliderer med den. Den andre *power-up*'en er hastighet som gjør at avataren løper fortere enn normalt. *Power-up* for hastighet er tidsbegrenset og gir en god dynamikk i forhold til det å være først til å levere ringen eller

flykte fra fiender. Hvis spilleren ligger bak andre spillere kan ha finne hastighets *power-ups* for å innhente forspranget.

4.1.1 Nettverk og kommunikasjon

Vi greide å sette oss inn i Netcode for GameObjects 1.2.0 og forstå hvordan vi skulle synkronisere data over nettet.

Vi benyttet oss av RPCs og NetworkVariables for å oppnå kommunikasjon mellom klient og server. Å oppnå kommunikasjon mellom klient og server var en av nøkkelpunktene i oppgaven fra vår oppdragsgiver, samtidig som vi skulle fokusere på kommunikasjon mellom ulike klienter.

Riktig plattform blir detektert under oppstart og følgende konfigurasjoner er testet mellom fysiske enheter.

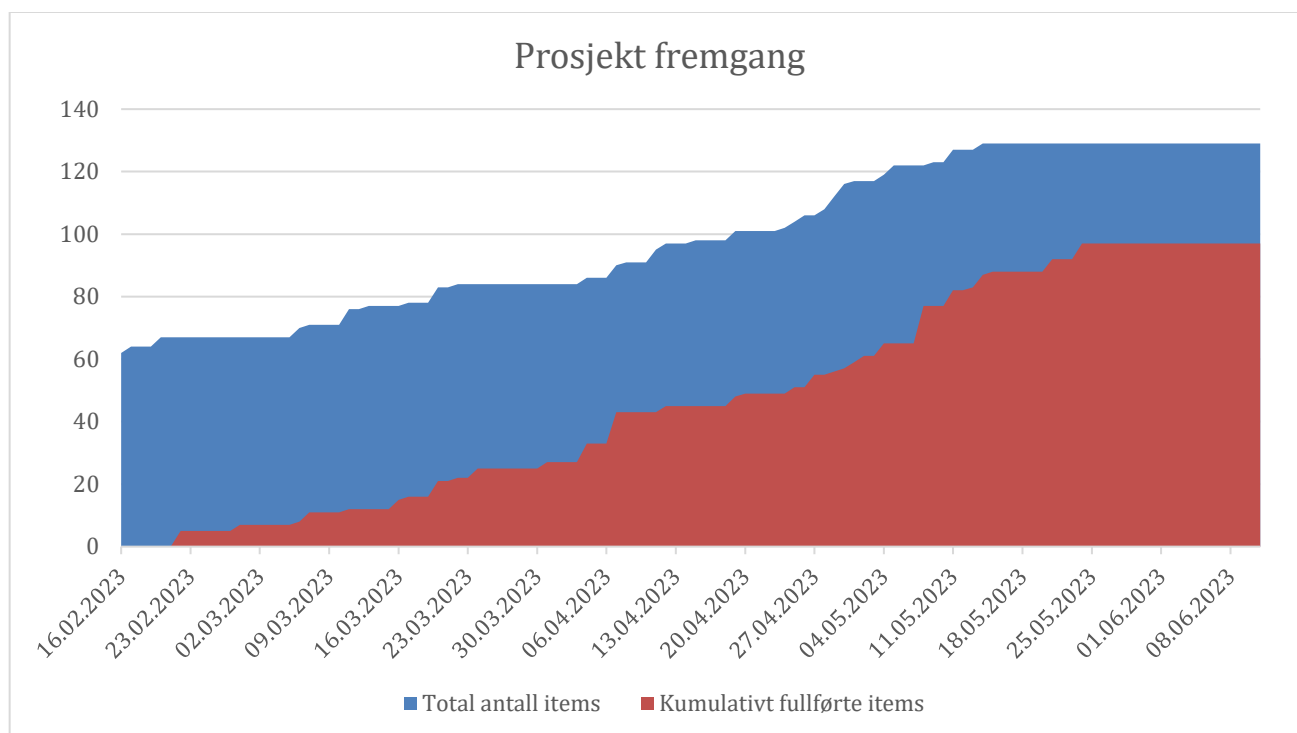
- Windows (host) – Windows (klient)
- Android (host) – Windows (klient)
- Windows (host) – Android (klient)
- Android (host) – Android (klient)

Med Android fungerer spillet på enheter med 64-bit prosessorer.

Vi har også signert .apk filen for å gjøre installasjonen på mobil enklere.

På Trello har vi dedikert en egen kolonne til synkronisering [20]

4.2 FREMGANG GJENNOM UTVIKLINGEN



Figur 12 Fremgangen i utviklingen var forventet å være en jevn linje, men det ble mange hopp.

			Ukenummer																						
Aktivitet	Dato start	Dato slutt	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Forprosjektrapport	01.01.2023	01.02.2023																							
Kravspesifikasjon	01.02.2023	15.02.2023																							
Online Multiplayer	15.02.2023	21.05.2023																							
Chat	15.03.2023	21.05.2023																							
Kryssplattform	01.04.2023	21.05.2023																							
Videre design/finpusning	08.04.2023	21.05.2023																							
Sluttrapport	15.05.2023	06.06.2023																							
Sluttpresentasjon	05.06.2023	11.06.2023																							

Tabell 2 Fremgangen i utviklingen vist ukevis.

4.3 NETTVERKSYTELSE

K. Raaen og T. M. Grønli refererer til en artikkel av Claypool, M. og Claypool, K som oppgir følgende, "[...]limits are estimated at 100ms for "first person avatar" games, 500ms for "third person avatar" games, and 1000ms for "omnipresent" games." [10]

Vårt spill er i tredje person og er PVE med en begrensning på 4 spillere per server og bør angivelig være under 500 ms for å gi en akseptabel opplevelse. De kategoriene vi skiller oss mest fra er spill som er sensitive ovenfor timing og raske reaksjoner. Fortrinnsvis racing og første person PVP der konkurranse er sentralt.

Claypool, M., og Claypool, K. belyser at det varierer hva et online spill krever av ytelse i forhold til type interaksjon.

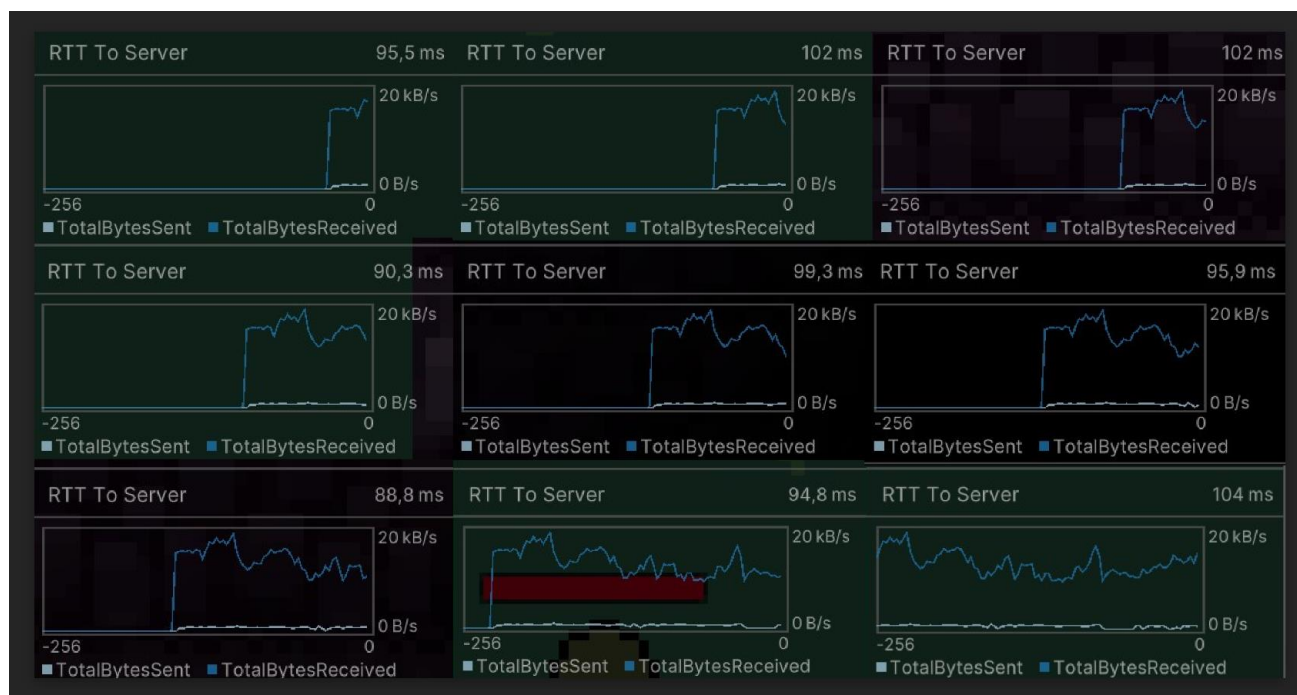
Across all genres, player actions vary along two primary axes, deadline and precision. Deadline is the time required to complete the action, that is the length of time it takes to achieve the final outcome of the action. [...] Precision is the degree of accuracy required to complete the interaction successfully. [1, s. 218]

Vi kan ikke påstå at vi har et spill som krever mye i forhold til begrepet «precision». Det er tilstrekkelig at spilleren kan nå fienden med sverdet, trykker space for å angripe, og treffer hvor som helst innenfor capsule collideren som omgir fienden for å få registrert treff. Aktiviteten er heller ikke tidssensitiv. Vi kjører et «owner» autoritativt oppsett på input fra spilleren. Fiendene får skade uansett hvis spilleren treffer og det blir formidlet til alle klienter, likevel om det er litt forsinkelse. Utfallet virker rettferdig og spillerne kan hjelpe hverandre med å nedkjempe fiender.

NetCode henviser til wikipedia og oppgir en generell grense på 200 ms for at spillere opplever forsinkelse [24]

Vi overvåket trafikk i løpet av en sesjon for å få et inntrykk av hvilken ytelse spilleren kan forvente. Figur 15 viser nettverkstrafikk på en Windows-klient der spillerne bor i forskjellige deler av landet. 5015 bergen, 7200 Kyrksæterøra og 8514 Narvik.

Det er tatt bilder med jevne mellomrom gjennom spillsesjonen. Trafikken nøydde sitt høyeste rundt 20 kb/s, mens det vanligste er et sted mellom 10 kb/s og 20 kb/s. I tillegg målte vi Round Trip Time(RTT).



Figur 13 Nettverkstrafikk hos klienten under en gjennomspilling.



Bilde 19 Vi får en spike når mange fiender er i nærheten av spilleren

Vi fulgte med når aktiviteten var størst. Aktiviteten var størst og toppet ut når det var flest fiender/NPCs i nærheten. Vi kan enkelt se når innslaget av fiender kommer midt på bilde 19

Et bevisst tiltak vi gjorde tidlig for å motvirke forsinkelser er å bruke C# events foran Unity events [25]

5 DISKUSJON

5.1 PROSJEKTETS AVGRENSNING

En bekymring var at prosjektet ikke hadde tilstrekkelig avgrensning. Vi hadde planlagt mye i forprosjektet og kanskje strakk vi oss langt utover kravene oppgaven skisserte. Dette reflekteres senere i dette kapittelet i forhold til en god del funksjonalitet som har blitt tatt vekk.

Det var en prototype oppdagsgiveren ønsket. En prototype er et fokusert produkt i tidlig fase uten noe annet formål enn å demonstrere en idé eller et konsept.

Samtidig er Unity en spillmotor som legger til rette for utvikling på et høyt nivå. Det kan innvendes at vi burde kunne utrette mer med et verktøy der vi får en del gratis funksjonalitet og automatikk. Å skrive kode er en stor del av arbeidet, men vi burde få gjennomført mer med mindre kode sammelignet med en gruppe der tilnærmet alt er kode. Det gjorde også at vi satte oss mange mål.

5.2 DEFINISJON AV SPILL

Det sto i oppgavebeskrivelsen at vi skulle utvikle en prototype på et spill. Det er da nærliggende å stille spørsmål ved hvor grensen går for å oppnå definisjonen av, ikke bare en prototype på et spill, men også et spill i seg selv. Er det fortsatt et spill hvis vi må skjære drastisk ned på vanlig funksjonalitet som er vanlig i spill? Men da beveger vi oss inn på filosofiens territorium og vi overlater til brukeren å vurdere i hvilken grad prototypen kvalifiserer til å være et spill.

5.3 PLANLEGGING

Planleggingen kunne vært mer tilstrekkelig detaljert i oppstarten. Vi hadde de store linjene, men det var elementære problemstillinger vi ikke besvarte. Derfor ble vi nødt til å inkludere oppgaver etter hvert som vi behovet meldte seg. Eksempelvis burde vi ha vurdert en GameManager fra starten av. Den overser spill-logikk fra et sentralisert ståsted og tar seg av de globale forandringene, som å avslutte spillet og å informere alle om at spillet er vunnet. Den ble implementert sammen med flere managere etter hvert. Ikke minst manglet planen en oversikt over innhold som skulle synkroniseres over nett.

Det ble kontinuerlig lagt til flere oppgaver underveis når vi innså at oppgavene vi laget i starten ikke var tilstrekkelig. Det skapte avhengigheter mellom oppgaver, og når disse ble gjennomført ble de andre ferdige samtidig. Vi hadde for mange systemer og for høye ambisjoner i forhold til oppgaven. Dette gjenspeiler seg i figur 12.

5.4 AVVIK FRA OPPRINNELIG PLAN OG OPPSATTE MÅL

I kapittel 2 understreket vi nødvendigheten av tilfredsstillende spillopplevelse. Vi mente at en prototype bør fokusere på spillopplevelse. Konsekvensen var som nevnt at vi nedprioriterte funksjoner som ikke var direkte knyttet til spillopplevelse. Vi redegjør her for de funksjonene vi bevisst har valgt bort.

5.4.1 Åpnings-scene

Det var planlagt å ha enda en scene i begynnelsen av spillet. En scene der spillerne fikk møte karakteren som ga oppdraget om å finne den tapte ringen. Dette skulle foregå i en landsby. Men fordi det i hovedsak dreide seg om introduksjon av bakgrunnshistorien og ikke spillopplevelse, valgte vi å fjerne denne scenen og isteden bruke ett enkelt stillbilde før spillet starter. Bildet gir samme introduksjon, men uten et interaktivt miljø.

5.4.2 Lobby

Det viste seg å være en større oppgave å bruke Unity sitt LobbyService API enn først antatt. Vi hadde flere dager med kun lesing av dokumentasjon og utprøving av enklere tester. Vi gikk så i gang med å utvikle vår egen implementasjon. Systemet er komplisert i den forstand at det krever mange sjekker på at tilkoblinger er gjort på riktig måte og synkroniseres i riktig rekkefølge før man får en tilfredsstillende opplevelse. Det krevde også at vi bygde opp vår kodebase som holder styr på den lokale delen av lobbyen kontra informasjonen som blir lagret på nett. I praksis var det mange prøvinger og feilinger før vi fikk det til å fungere som det gjør i vår ferdige prototype.

Lobbysystemet vårt kunne også hatt mer funksjonalitet: Spillere kan ikke koble seg opp mot eksisterende lobby med lobbykode. Spillere kan heller ikke se en oversikt over tilgjengelige lobbyer for så å velge å bli med i en.

Det er verdt å nevne at en lobby ikke var med i planleggingsstadiet. Selv om de manglende lobbyfunksjonene kunne bedret spillopplevelsen, valgte vi derfor å forskyve disse mot slutten av prosjektet for å fokusere på andre problemer. Til slutt valgte vi disse funksjonene bort med hensyn til tid.

Når det kommer til Unity sin LobbyService har vi oppdaget ulemper med den i seg selv. Den største av disse er at APIet ikke er event-drevet. I stedet krever det polling for oppdateringer, noe som innebærer at programmet regelmessig må sjekke serveren for ny informasjon. Denne polling-prosessen kan generere unødvendig nettverkstrafikk og lokale oppdateringer, som både kan være belastende for serveren og lokal maskinvare. En annen ulempe er at LobbyService kan være unødvendig stort i forhold til de funksjonene man trenger. Et alternativ til det kan være å bygge en egen lobbytjeneste rundt Unity sitt Netcode API.

Mot slutten av prosjektet viser deg seg at Unity har oppdatert LobbyService til å være event-drevet. Dette viser igjen at vi bruker rammeverk som er i rask utvikling og derfor krever en del tilpassning. Bruker «gladys_khk» fra Unity Technologies bekrefter i slutten av 2022 at event-dreven Lobby vil bli lansert i løpet av første kvartal av 2023 [27].

Til tross for disse ulempene, er det store fordeler ved å benytte seg av et allerede utviklet API. I tillegg er det mulig å bygge på toppen av LobbyService for å opprette et tilpasset event-system. Vi valgte på grunn av tidsforbehold å ikke bygge mer på toppen av tjenesten enn vi strengt tatt trengte, for å få den til å fungere.

5.4.3 Poengsystem

Poengsystemet ble valgt helt bort da vi forstod at vi trengte den resterende tiden til å prøve å få de mer grunnleggende elementene på plass. Det var et vanskelig valg å utelate dette fordi det kunne ha gitt spillet et løft og bidratt til ytterligere spillbarhet.

Poeng ville gitt spillet et løft i form av vennskapelig konkurranse. Spilleren må ta et valg der han vet han får mye poeng for å hjelpe avataren til en annen spiller opp. I dette ligger det at eieren(spilleren) av avataren som blir brakt til live, igjen har muligheten til å tjene poeng. Samtidig blir spillet tryggere ved at flere kan kjempe sammen mot fiender. På den andre siden kan en spiller velge å ikke hjelpe avataren til en annen spiller. Da kan spilleren med avataren som er i live kjøre solo og unngå å potensielt å bli forbigått i poengsum hvis han hjelper noen som ligger nede. Det skulle også være poeng for andre bragder, men langt mindre enn den vi nettopp beskrev.

5.4.4 Lyd

Lyd er relativt enkelt å legge til i Unity. Å komponere og mikse lyd i 2 dimensjoner er en annen sak. Ingen av oss følte oss komfortable med å gjøre dette. I lys av at det er en prototype valgte vi derfor å ikke ta med lyd i spillet.

5.4.5 Koordinater i chat

Vår opprinnelige plan var at det i chat-vinduet skulle vises koordinater til spillere som har funnet noe av interesse, eller inngangen til gruva. Vi erstattet dette med en enkel meldingstjeneste uten koordinater og vurderer at det er tilstrekkelig for prototypen.

5.4.6 Kompass

Vi hadde planlagt å ha med en kompass-pill øverst på skjermen som viser retningen til en annen spiller som har funnet noe. Denne oppgaven ble for langt nede i prioriteringsrekkefølgen til at vi rakk å implementere den.

5.4.7 Stålsverd

Stålsverd var planlagt fra start, men ble ikke med på grunn av tidsmangel.

5.4.8 Usynlighet på avatarer

Den opprinnelige planen var at en avatar først skulle bli synlig for de andre når han har funnet objekter eller inngangen til gruva. Dette avhenger av poengsystem for å ha riktig dynamikk og tjener ikke formålet når det er en del av oppgaven at avatarer skal være synlige for hverandre. Det ville også fjernet

problematikk for synkronisering i starten av spillet og dermed tatt vekk fra mye av poenget oppgaven. Vi valgte derfor denne funksjonaliteten helt bort, uavhengig av vanskelighetsgrad eller tidsbruk.

5.4.9 Pause spill ved møte med objekter

Det var i starten planlagt at vi skulle påtvinge spillere pause når de finner objekter av nytte (power-ups, fakkel, ring) for å balansere mot spillere som ikke har funnet noe. Det vil mest sannsynlig virke irriterende å bli tvunget til en pause i noen sekunder uten en god forklaring.

5.4.10 Kunstig intelligens

Vi fant det utfordrende å implementere en form for intelligent navigasjon på fienden eller *NPC*. Vi brukte en del tid på det, men uten å komme i mål. Vi nevnte noen tilnærmer i kap. 2.2 I etterkant har vi innsett at algoritmer i forhold til *AI*-navigasjon krever bedre planlegging og tilpasning av områder på forhånd.

Vi endte isteden opp med en løsning riktignok simulerer adferd, men det er bare en enkel state-machine som ikke er relatert til *AI*. Det er en random-generator som gjør at *NPCs* beveger seg i tilfeldige retninger på tilfeldige tidspunkt. Hvis en *NPC* treffer en hindring, velger den en annen retning etter et tilfeldig antall sekunder.

5.5 KJENTE BUGS OG PROBLEMER

5.5.1 Lobby

Vi har noen kjente bugs i lobbysystemet vårt:

1. Lobby som har vært aktiv over for lang tid tar ikke imot spillere lenger, til tross for kontinuerlige heartbeat oppdateringer.
2. I lobbyen vil man i enkelte tilfeller få opp duplikater av spillere som først har forlatt lobbyen for så å returnere.

Problem 1 kan ha med synkronisering å gjøre, men det kan også være noe i *LobbyService* som ikke fungerer tilfredstillende. Problem 2 kan sannsynligvis løses med en logisk sjekk under oppdatering av visuell fremvisning av spillere. Vi har ikke hatt tilstrekkelig med tid til å gå videre inn i disse problemene.

5.5.2 Lys

Når flere spillere er i nærheten av hverandre ser de hverandres lys, men lyset har ingen grense for hvor sterkt det kan bli. Lysene adderes og overstiger dermed en normal lyssetting, noe som blant annet gjør at fargene hviskes ut. Dette kan muligens løses i lysets innstiller i Unity.

5.5.3 Animasjon

Enda en ulempe er at i et forsøk på å bruke mindre båndbredde, så ble det lagd flere bilder for bruk i animasjon i stedet for å bruke RPC-kall til å fortelle avatarer til å snu seg mot høyre og venstre.

Dette har i etterkant vist seg å ikke være en så god løsning, fordi hvis det skal jobbes med animasjoner i framtiden, så må det lages enda flere bilder som tar mere tid å lage.

Det første som burde gjøres før det arbeides videre med animasjoner, er at spillernes og fiendenes C# skript skrives om til å bruke RPC-kall til å snu på avatarene, og fjerne overflødige bilder fra animator-kontrollerne til spillerne og fiendene.

5.5.4 Fiende

Et vedvarende problem har vært å få fienden å stoppe å angripe når avataren ligger nede, ta en pause og gå tilbake til *State.Roaming*. Det virker til en viss grad, men hvis fienden skifter target flere ganger kan

det oppstå en konflikt og fienden fortsetter å løpe mot og angripe avataren etter at den er nedkjempet. Vi har ikke klart å identifisere nøyaktig hva problemet er, men det åpenbart at fienden ikke mister target etter at avataren er nede. Vi kan skru av Circle Collideren og der ser ut til å virke, men det har en uheldig sideeffekt; Fiender som har satt jaktet på avataren tidligere men gått tilbake til State.Roaming, stopper også opp. Fiendene går heller ikke tilbake til State.Roaming hvis vi skrur av Circle Collideren.

5.5.5 Chat-input for Android

Det er et minus at chat-input ikke er optimalisert for mobil. Når spilleren trykker i chatten okkuperer tastaturet det meste av skjermen. Får å komme ut av chatten hjelper det ikke å trykke 'ok' eller hake-symbolet. Spilleren må trykke i spill-vinduet for å komme tilbake til spillet.

5.5.6 Avslutning av spill

En åpenbar ulempe med spillet i sin nåværende tilstand er avhengigheten av at host ikke avslutter spillet. Hvis den som er host avslutter spillet, dvs. stenger ned serveren, vil naturligvis spillet stoppe for alle klienter. Hvis vi hadde hatt en sentralisert server ville ikke dette være et problem.

Forslag til noen løsninger

- Spill design: Frigi kamera til host når han er død eller gi få kamera til å følge etter annen spiller
- Spill design: Endre målet med spillet. Forsvare, passe på lederen av gruppa, fordi det er han som har mistet ringen og må vise spillerne hvor den er. Krever endring av bakgrunnshistorie.
- Teknisk løsning: Host migration. En klient overtar rollen som host. [26]
- Teknisk løsning: Porte spillet til dedikert spill server. Den virker uavhengig av hvem av spillerne som kobler fra.

5.6 GRAFIKK

En annen feilkalkulering var at vi begynte med et uteområde med alt for store dimensjoner. Det tok hele tre minutter å gå med avataren fra en ende til en annen. Etter hvert ble det tydelig at vi måtte foreta en drastisk reduksjon av området for at innholdet skulle være håndterbart. Området skulle fylles med innhold. Det var lite hensiktsmessig å la avatarene løpe gjennom store tomme områder. Til slutt landet vi på en fornuftig størrelse vi mener passer spillet godt og balanserer i forhold til underholdningsverdi. Det var positivt at vi fant ut av dette før vi designet gruveområdet slik at vi kunne bruke samme størrelse.

Selv om det ble brukt 16-biters PNG-bilder til grafikk, krevde det mye redigering for å kunne bruke de. Avataren hadde opprinnelig et dyrehode som måtte gjøres om til et menneskehode. Den hadde også en kappe som dekket mye av avatarens kropp som måtte fjernes. Kroppen måtte tegnes nesten fra bunnen av der kappen dekket avataren.

5.7 NETTVERSKOMMUNIKASJON

Vi måtte foreta et mentalt paradigmeskifte for å forstå hva det betyr å utvikle multiplayer i forhold til singleplayer. I det øyeblikket en klasse arver fra NetworkBehaviour må vi tenke annerledes fordi alle klienter i tillegg til server leser samme kode. Da må vi styre tillatelser i koden og avgjøre om server har full autoritet eller om klientene skal få autoritet til å utføre noe.

Angående nettverk har vi ikke merket at spillet er påvirket lav overføringshastighet. Hvis målingene vi presenterte i forrige kapittel er representative er vi godt fornøyde. En RTT på rundt 100 ms er på nivå med estimerte krav til FPS og racing spill. I Uniy er det mulig å simulere forsinkelse og vi prøvde med 500 ms forsinkelse, men det så ikke ut til å påvirke opplevelsen.

6 KONKLUSJON OG VIDERE ARBEID

Vår erfaring gjennom dette prosjektet understreker betydningen av spill-verktøy i spillutviklingsprosessen. Vi brukte omtrent to måneder på å planlegge prosjektet, og ytterligere tre måneder på programmering og utvikling. Som tre dataingeniørstudenter med minimal erfaring innen spillutvikling, har vi delvis lyktes i å utvikle en fungerende prototype på et flerspiller, kryssplattform nettspill i løpet av fem måneder.

Det var tydelig at spill-verktøyene var avgjørende for å fremskynde utviklingsprosessen. Uten slike verktøy, ville vi ha måttet programmere på et langt lavere nivå. Dette ville ha lagt til betydelig kompleksitet og tid til prosjektet, spesielt når det gjelder nettverkssynkronisering.

Det har allikevel vært et omfattende rammeverk å sette seg inn i. Vi har vært nødt til å studere dette i detalj før vi i det hele tatt var i stand til å få nettverksdelen til å fungere.

Selv om det er vanskelig å fastslå hvor langt vi kunne ha kommet med en mer lavnivå tilnærming, vurderer vi at vi har oppnådd vårt mål om å utvikle en spillbar prototype. Vår prototype inneholder grunnleggende elementer som tillater flerspiller, kryssplattform spill over nettet.

Vi har utelatt en rekke elementer, og dette reiser spørsmålet om hva som egentlig definerer en "spill-prototype". Dette er et omfattende spørsmål som vi ikke kan svare fullt ut i denne oppgaven. Likevel mener vi at vår prototype har nok funksjonelle elementer til å fastslå at det er et levedyktig konsept som kan videreutvikles.

Basert på tidligere diskusjon, med tanke på videre utvikling, anbefaler vi å først å løse de logiske feilene. Deretter er det aktuelt å bygge et poengsystem og gi avatarene evnen til å hjelpe hverandre opp igjen innenfor en bestemt tidsramme. Den største uttellingen av poeng ville være hvis en spiller hjalp avataren til en annen opp igjen. Det ville styrket samhandling, forsvare valg av nettverkstopologi på grunn av mulighet for ytterligere samarbeid, og gi mer dynamikk og mening med spillet.

Til slutt vil vi konkludere med at våre funn demonstrerer at spill-verktøy har en betydelig innvirkning på utviklingshastigheten og -kvaliteten, selv for utviklere med minimal tidligere erfaring. Det demonstrerer også hvor langt vi kom i prosessen og hvilke aspekter vi ble nødt til å kutte. Videre forskning kan kanskje utforske hvilke verktøy som er mest effektive, og hvordan de best kan brukes i forskjellige utviklingssituasjoner.

7 REFERANSER

- [1] M. Claypool, K. Claypool, "Latency Can Kill : Precision and Deadline in Online Games," presentert på Proceedings of the First ACM Multimedia Systems Conference, New York, NY, USA, 2010. DOI: 10.1145/1730836.1730863, hentet fra: <https://dl-acm-org.mime.uit.no/doi/pdf/10.1145/1730836.1730863>
- [2] Wikipedia, The Free Encyclopedia, [Online]. Hentet fra https://en.wikipedia.org/wiki/The_Legend_of_Zelda:_A_Link_to_the_Past (Lastet ned 04.06.2023).
- [3] Wikipedia, The Free Encyclopedia, [Online]. Hentet fra https://en.wikipedia.org/wiki/Chrono_Trigger (Lastet ned 04.06.2023).
- [4] E. Adams, *Fundamentals of Game Design*. 2.utgave. Berkley, CA, USA: Pearson Education, 2010.

- [5] C. Fabricatore «Gameplay and game mechanics design: a key to quality in videogames», presentert på OECD-CERI Expert Meeting on Videogames and Education, DOI:10.13140/RG.2.1.1125.4167, hentet fra <https://www.oecd.org/education/ceri/39414829.pdf>
- [6] M. Killick, *The Way We Play: Theory of Game Design*, 1. utg. Cambridge, UK: Apress, 2022. Tilgjengelig: <https://link-springer-com.mime.uit.no/book/10.1007/978-1-4842-8789-7> (Lastet ned 04.06.2023).
- [7] Wikipedia, The Free Encyclopedia, [Online]. Hentet fra https://en.wikipedia.org/wiki/Arrow_keys (Lastet ned 04.06.2023).
- [8] Adam Telfer. «Touch Control Design: Ways of Playing on Mobile.» Mobilefreetoplay.com. Hentet fra: <https://mobilefreetoplay.com/control-mechanics/> (Lastet ned 04.06.2023)
- [9] Android Tools, Youtube. *Diablo Immortal Mobile Gameplay - Official Launch*. (Juni 2, 2022). Sett: Juni.06.2023. [Online video]. Hentet fra: <https://youtu.be/WltSrmtz254>
- [10] K. Raaen and T. M. Grønli, «Latency Thresholds for Usability in Games: A Survey. » Presentert på Norsk Informatikkonferanse, Høgskolen i Østfold, Norway 27. October 2014, [Online]. Hentet fra: <https://ojs.bibsys.no/index.php/NIK/article/view/9/6> (Lastet ned: 04.06.2023)
- [11] Unity Technologies. «Tricks and patterns to deal with latency.» Docs-multiplayer.unity3d.com. Hentet fra <https://docs-multiplayer.unity3d.com/netcode/current/learn/dealing-with-latency/#authority> (Lastet ned: 04.06.2023)
- [12] Unity Technologies. «Tricks and patterns to deal with latency.» Docs-multiplayer.unity3d.com. Hentet fra <https://docs-multiplayer.unity3d.com/netcode/current/learn/dealing-with-latency/#client-authority> (Lastet ned: 04.06.2023)
- [13] Unity Technologies. «Relay servers.» Docs.unity.com. Hentet fra <https://docs.unity.com/relay/en/manual/relay-servers> (Lastet ned: 05.06.2023)
- [14] Unity Technologies. «Create a game with a listen server and host architecture.» Docs-multiplayer.unity3d.com. Hentet fra <https://docs-multiplayer.unity3d.com/netcode/current/learn/listen-server-host-architecture/> (Lastet ned: 04.06.2023)
- [15] Unity Technologies. «Sending Events with RPCs.» Docs-multiplayer.unity3d.com. Hentet fra: <https://docs-multiplayer.unity3d.com/netcode/current/advanced-topics/messaging-system/> (Lastet ned: 04.06.2023)
- [16] Zdravko Jakupec. «Saving Data Between Scenes in Unity.» Sitepoint.com. Hentet fra: <https://www.sitepoint.com/saving-data-between-scenes-in-unity/> (Lastet ned 04.06.2023)
- [17] BankMyCell. «Android vs. Apple Market Share: Leading Mobile Operating Systems (OS).» Bankmycell.com. Hentet fra: <https://www.bankmycell.com/blog/android-vs-apple-market-share/> (Lastet ned 04.06.2023)
- [18] Newzoo, «Global Games Market Report», Amsterdam, North Holland, 2023. [Online]. Hentet fra: https://resources.newzoo.com/hubfs/Reports/Games/2022_Newzoo_Free_Global_Games_Market_Report.pdf (Lastet ned 04.06.2023)
- [19] Wikipedia, The Free Encyclopedia, [Online]. Hentet fra: https://en.wikipedia.org/wiki/Write_once,_run_anywhere

- [20] Trello. «Multiplayer online spill.» Trello.com. Hentet fra:
<https://trello.com/invite/b/KWXXfPl3/ATTId40c87a822360b34dce631ba311483066F17B70E/multiplayer-online-spill> (Lastet ned 04.06.2023)
- [21] UheartBeast. «Action RPG Resources.zip» Github.com. Hentet fra:
<https://github.com/uheartbeast/youtube-tutorials/blob/87eba5bf8f7796029c39033b107b064e2969bbad/Action%20RPG/Action%20RPG%20Resources.zip> (Lastet ned 04.06.2023)
- [22] LukeThePolice. «Pixelarium 16x16.» Lukepolice.itch.io. Hentet fra:
<https://lukepolice.itch.io/pixelariumgrasslands> (Lastet ned 04.06.2023)
- [23] Aurusenth. «Shroom Cave Asset Pack.» Aurusenth.itch.io. Hentet fra:
<https://aurusenth.itch.io/shroom-cave-asset-pack> (Lastet ned 04.06.2023)
- [24] Unity Technologies. «Latency and Packet Loss.» Docs-multiplayer.unity3d.com. Hentet fra:
<https://docs-multiplayer.unity3d.com/netcode/current/learn/lagandpacketloss/> (Lastet ned 04.06.2023)
- [25] Jackson Dunstan. «Event Performance: C# vs. UnityEvent.» Jacksondunstan.com. Hentet fra:
<https://www.jacksondunstan.com/articles/3335> (Lastet ned 04.06.2023)
- [26] Unity Technologies. «Host migration.» Docs.unity.com. Hentet fra:
<https://docs.unity.com/lobby/en/manual/host-migration> (Lastet ned 04.06.2023)
- [27] Unity Tehnologies. «Question: Realtime Updates with events.» Forum.unity.com. Hentet fra:
<https://forum.unity.com/threads/realtime-updates-with-events.1317909/> (Lastet ned 04.06.2023)

8 LISTE OVER FIGURER

Figur 1-5: Nettverkstopologier	15
Figur 6: Relay servers	20
Figur 7: Diagram over flyt i meny	24
Figur 8: Diagram for lobbysystem	25
Figur 9: Overgang mellom scener	30
Figur 10: Sceneovergang med GameManager	30
Figur 11: Spawning av objekter over nettverk	32
Figur 12: Prosjekt fremgang	37
Figur 13: Nettverkstrafikk hos klient	39

9 LISTE OVER TABELLER

Tabell 1: Tabell over menysider	22-23
Tabell 2: Fremdriftsplan	38

10 LISTE OVER AKRONYMER OG FORKORTELSER

4G - Fjerde generasjon mobilnett-teknologi
API - Application programming interface
Apk - Android Package Kit
DGS - Dedicated game server
DTLS - Datagram Transport Layer Security
FPS – First person shooter
GUI – Graphical User Interface
iOS - iPhone operating system
MacOS - Macintosh Operating System
MMO – Massive Online Multiplayer
NPC - Ikke-spiller karakter
Open Source – Kildekode tilgjengelig for offentligheten
PVE - Player versus Environment
PVP – Player versus player
PNG - Portable Network Graphic
RPC - Remote Procedure Call
RTT - Round-trip delay time
UDP - User Datagram Protocol
WASD – W, A, S og D -tastene på tastaturet
WSS - WebSocket Secure

11 VEDLEGG

Vedlegg A: Forslag til bacheloroppgave V23, «Prototype på kryssplattform flerspiller online spill»

Vedlegg B: Klasse-avhengigheter

Vedlegg C: Tilstands-maskin for fiende

Vedlegg D: Dokumentasjon av kode

Vedlegg E: Spill-instruksjoner

Mobilbuild for Android: JourneyOfTheHeroes.apk

Windowsbuild: Journey of the heroes-WindowsBuild.zip

Prosjektfiler: hovedoppgave-gruppe3-UIT-2023-main.zip